TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN THANH TIỀN – 524H0035
LÊ HOÀNG MINH KHÔI - 524H0102

TỐI ƯU TẢI, LƯU TRỮ ĐỆM VÀ HIỂN THỊ HÌNH ẢNH TRONG ỨNG DỤNG ANDROID

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



NGUYỄN THANH TIỀN - 524H0035
LÊ HOÀNG MINH KHÔI - 524H0102

TỐI ƯU TẢI, LƯU TRỮ ĐỆM VÀ HIỂN THỊ HÌNH ẢNH TRONG ỨNG DỤNG ANDROID

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN ỨNG DỤNG DI ĐỘNG

Người hướng dẫn
TS. Lê Văn Vang

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn **TS. Lê Văn Vang**, giảng viên hướng dẫn, đã tận tình giảng dạy, hướng dẫn và cung cấp những kiến thức cần thiết trong suốt quá trình học tập và thực hiện đề tài. Những kiến thức và góp ý của Thầy là cơ sở quan trọng giúp chúng em từng bước nghiên cứu, xây dựng và hoàn thiện đề tài "Tối ưu tải, lưu trữ đệm và hiển thị hình ảnh trong ứng dụng Android".

Thông qua quá trình thực hiện đề tài, chúng em đã có cơ hội vận dụng các kiến thức đã học vào thực tế, đồng thời nâng cao hiểu biết về phát triển và tối ưu hiệu năng ứng dụng Android. Mặc dù đã cố gắng hoàn thiện báo cáo, chúng em khó tránh khỏi những thiếu sót và kính mong nhận được những nhận xét, góp ý từ Thầy để đề tài được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

TP. Hồ Chí Minh, ngày 01 tháng 11 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Tiền

(Nguyễn Thanh Tiền)

Khôi

(Lê Hoàng Minh Khôi)

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH
TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng em xin cam đoan đây là công trình nghiên cứu của chúng em và được thực hiện dưới sự hướng dẫn khoa học của **TS. Lê Văn Vang**. Các nội dung nghiên cứu và kết quả trình bày trong đề tài này là trung thực và chưa được công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét và đánh giá được chúng em thu thập trong quá trình nghiên cứu, thực nghiệm và từ các nguồn tham khảo có ghi rõ nguồn gốc.

Ngoài ra, trong đề tài có sử dụng một số nhận xét, đánh giá, tài liệu và số liệu từ các tác giả hoặc tổ chức khác. Các nội dung này đều được trích dẫn và chú thích nguồn đầy đủ theo quy định.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng em xin hoàn toàn chịu trách nhiệm về nội dung của đề tài. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm về tác quyền, bản quyền do chúng em gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 01 tháng 11 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

TIỀN

(Nguyễn Thanh Tiền)

KHÔI

(Lê Hoàng Minh Khôi)

MỤC LỤC

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT 4

1.1 Tổng quan về tải và hiển thị hình ảnh trên Android 4

1.2 Tải hình ảnh bất đồng bộ 4

1.3 Cơ chế lưu trữ đệm hình ảnh 5

1.3.1 Memory Cache 5

1.3.2 Disk Cache 6

1.3.3 HTTP Cache và chính sách Cache 6

1.4 Bitmap, giải mã ảnh và quản lý bộ nhớ 7

1.4.1 Bitmap và giải mã hình ảnh 7

1.4.2 Định dạng hình ảnh 8

1.4.3 Quản lý bộ nhớ 9

1.5 Lazy Loading và tối ưu hiển thị danh sách 9

1.6 Các kỹ thuật tối ưu hình ảnh 10

1.6.1 Resize và Downsampling 10

1.6.2 Thumbnail 10

1.6.3 Placeholder và Error Handling 10

1.6.4 Kết hợp các kỹ thuật tối ưu 11

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ ỨNG DỤNG 12

2.1 Phân tích yêu cầu ứng dụng 12

2.2 Kiến trúc ứng dụng 13

2.3 Thiết kế luồng tải và lưu trữ đệm hình ảnh 14

2.4 Thiết kế Baseline và Optimized 15

2.5 Thiết kế kịch bản thực nghiệm 16

CHƯƠNG 3. CÀI ĐẶT VÀ TỐI ƯU ỨNG DỤNG 17

3.1 Môi trường và công nghệ phát triển 17

3.2 Cài đặt các chức năng của ứng dụng 17

3.2.1 Danh sách và màn hình chi tiết 17

3.2.2 Trạng thái tải, lỗi ảnh và xử lý cuộn 18

3.3 Cài đặt mô-đun tải ảnh và Cache 19

3.4 Xây dựng hai phiên bản Baseline và Optimized 21

3.5 Quản lý mã nguồn và hướng dẫn Build 22

3.6 Kết quả cài đặt 23

CHƯƠNG 4. PHƯƠNG PHÁP VÀ KỊCH BẢN THỰC NGHIỆM 24

4.1 Môi trường và công cụ thực nghiệm 24

4.2 Các chỉ số và tiêu chí đánh giá 24

4.3 Quy trình thực nghiệm 26

4.4 Các trường hợp kiểm thử 27

4.4.1 TC01 – First Load và TC02 – Cache Hit 29

4.4.2 TC03 – Fast Scroll và TC04 – High Resolution 29

4.4.3 TC05 – Offline và TC06 – Error/Slow Network 29

4.4.4 TC07 – Dataset Scaling và TC08 – Before/After Comparison 30

CHƯƠNG 5. KẾT QUẢ VÀ PHÂN TÍCH HIỆU NĂNG 31

5.1 Kết quả các trường hợp kiểm thử 31

5.1.1 TC01 – First Load 31

5.1.2 TC02 – Cache Hit 32

5.1.3 TC03 và TC04 – Fast Scroll và High Resolution *33*

5.1.4 TC05 và TC06 – Offline và xử lý lỗi *35*

5.1.5 TC07 – Dataset Scaling *36*

5.2 So sánh Baseline và Optimized 37

5.3 Phân tích tác động của các kỹ thuật tối ưu 40

5.4 Đánh giá tổng hợp và thảo luận 40

CHƯƠNG 6. SẢN PHẨM VÀ KẾT LUẬN 42

6.1 Sản phẩm hoàn thiện 42

6.2 Hướng dẫn cài đặt và sử dụng 43

6.3 Hướng dẫn tái hiện thực nghiệm 44

6.4 Kết quả đạt được và hạn chế 45

TÀI LIỆU THAM KHẢO 46

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1 Tổng quan về tải và hiển thị hình ảnh trên Android

Hình ảnh là một thành phần phổ biến trong các ứng dụng Android, đặc biệt đối với các ứng dụng thương mại điện tử, mạng xã hội, thư viện ảnh hoặc các ứng dụng có nội dung được tải từ Internet. So với các thành phần văn bản thông thường, hình ảnh thường có kích thước dữ liệu lớn hơn và cần trải qua nhiều bước xử lý trước khi có thể hiển thị trên giao diện.

Về cơ bản, quá trình hiển thị một hình ảnh từ Internet có thể được mô tả qua các bước: ứng dụng gửi yêu cầu tải ảnh, nhận dữ liệu hình ảnh, giải mã dữ liệu thành dạng có thể sử dụng trong bộ nhớ và cuối cùng hiển thị kết quả lên giao diện. Trong trường hợp hình ảnh đã được lưu trữ đệm, một số bước tải qua mạng có thể được bỏ qua, giúp rút ngắn thời gian hiển thị.

Khi số lượng hình ảnh tăng lên, quá trình này có thể tạo ra áp lực đáng kể lên tài nguyên của thiết bị. Việc liên tục tải dữ liệu qua mạng làm tăng thời gian chờ, trong khi giải mã nhiều hình ảnh có độ phân giải cao có thể làm tăng mức sử dụng bộ nhớ. Vì vậy, việc tối ưu quá trình tải, lưu trữ và hiển thị hình ảnh là cần thiết để đảm bảo ứng dụng duy trì khả năng phản hồi và trải nghiệm người dùng ổn định.

Trong đề tài này, ứng dụng được phát triển bằng Kotlin và Jetpack Compose, đồng thời sử dụng thư viện Coil để hỗ trợ quá trình tải và quản lý hình ảnh. Coil cung cấp các chức năng như tải ảnh bất đồng bộ, lưu trữ đệm trong bộ nhớ và bộ nhớ lưu trữ, thay đổi kích thước ảnh và quản lý vòng đời của các yêu cầu tải ảnh.

1.2 Tải hình ảnh bất đồng bộ

Một trong những nguyên tắc quan trọng khi phát triển ứng dụng Android là hạn chế thực hiện các tác vụ tốn nhiều thời gian trên Main Thread. Main Thread chịu trách nhiệm xử lý giao diện và tương tác của người dùng. Nếu các công việc như truy cập mạng hoặc giải mã hình ảnh lớn được thực hiện đồng bộ trên luồng này, giao diện có thể phản hồi chậm và xuất hiện hiện tượng giật, lag.

Do đó, quá trình tải hình ảnh nên được thực hiện theo cơ chế bất đồng bộ (asynchronous loading). Theo cơ chế này, các tác vụ tải và xử lý hình ảnh được thực hiện mà không chặn quá trình cập nhật giao diện. Sau khi hình ảnh được tải và xử lý hoàn tất, kết quả mới được chuyển đến thành phần giao diện cần hiển thị.

Trong Kotlin, Coroutine là một trong những cơ chế hỗ trợ lập trình bất đồng bộ. Các tác vụ có thể được phân chia theo mục đích, chẳng hạn Dispatchers.Main được sử dụng cho công việc liên quan đến giao diện, trong khi các tác vụ I/O như đọc dữ liệu hoặc truy cập mạng phù hợp hơn với Dispatchers.IO.

Trong phạm vi đề tài, thư viện Coil được sử dụng để thực hiện phần lớn quá trình tải hình ảnh. Coil là một thư viện tải ảnh dành cho Android và Compose, được xây dựng dựa trên Kotlin Coroutines. Thành phần AsyncImage của Coil cho phép tải hình ảnh từ URL và cập nhật giao diện theo trạng thái của yêu cầu mà không yêu cầu ứng dụng tự xây dựng toàn bộ cơ chế xử lý bất đồng bộ.

Bên cạnh đó, quản lý yêu cầu (request management) cũng có ý nghĩa khi người dùng cuộn nhanh qua danh sách. Một hình ảnh có thể không còn cần thiết khi item tương ứng đã rời khỏi vùng hiển thị. Việc hủy hoặc tạm dừng những yêu cầu không còn cần thiết giúp hạn chế công việc dư thừa và giảm mức sử dụng tài nguyên.

1.3 Cơ chế lưu trữ đệm hình ảnh

Cache là cơ chế lưu lại dữ liệu đã được truy cập để dữ liệu đó có thể được sử dụng lại mà không phải thực hiện toàn bộ quá trình tải và xử lý từ đầu. Đối với ứng dụng có nhiều hình ảnh từ Internet, cache có vai trò quan trọng trong việc giảm thời gian hiển thị, hạn chế yêu cầu mạng và cải thiện trải nghiệm khi người dùng xem lại nội dung.

1.3.1 Memory Cache

Memory Cache lưu các hình ảnh đã được xử lý gần đây trong bộ nhớ RAM. Ưu điểm chính của phương pháp này là tốc độ truy xuất nhanh. Khi ứng dụng cần hiển thị lại một hình ảnh còn tồn tại trong Memory Cache, hình ảnh có thể được lấy trực tiếp từ bộ nhớ thay vì tải lại qua mạng và thực hiện lại toàn bộ quá trình xử lý.

Tuy nhiên, RAM là tài nguyên giới hạn. Việc lưu quá nhiều hình ảnh hoặc lưu các Bitmap có kích thước lớn có thể làm tăng mức sử dụng bộ nhớ của ứng dụng. Vì vậy, Memory Cache cần có giới hạn và cơ chế loại bỏ những dữ liệu không còn phù hợp.

Một cơ chế phổ biến là Least Recently Used (LRU). Khi cache đạt đến giới hạn, các phần tử ít được sử dụng gần đây sẽ được ưu tiên loại bỏ để tạo không gian cho dữ liệu mới.

1.3.2 Disk Cache

Disk Cache lưu dữ liệu hình ảnh trên bộ nhớ lưu trữ của thiết bị. So với Memory Cache, việc đọc dữ liệu từ bộ nhớ lưu trữ chậm hơn đọc trực tiếp từ RAM nhưng có khả năng lưu trữ lượng dữ liệu lớn hơn và dữ liệu không phụ thuộc hoàn toàn vào vòng đời của bộ nhớ RAM.

Disk Cache đặc biệt hữu ích khi hình ảnh không còn tồn tại trong Memory Cache. Thay vì phải tải lại từ Internet, ứng dụng có thể sử dụng dữ liệu đã được lưu trên thiết bị. Điều này có thể làm giảm thời gian tải lại và lượng dữ liệu mạng cần sử dụng.

Trong Coil, ImageLoader có thể quản lý cả Memory Cache và Disk Cache. Việc kết hợp hai tầng cache cho phép ứng dụng ưu tiên nguồn dữ liệu có tốc độ truy xuất nhanh hơn trước khi thực hiện một yêu cầu mạng mới.

Có thể khái quát thứ tự tìm kiếm dữ liệu như sau:

Memory Cache → Disk Cache → Network

Nếu hình ảnh tồn tại trong Memory Cache, ứng dụng có thể sử dụng ngay dữ liệu trong RAM. Nếu không, hệ thống tiếp tục kiểm tra Disk Cache. Khi cả hai nguồn không có dữ liệu phù hợp, hình ảnh mới cần được tải từ nguồn mạng.

1.3.3 HTTP Cache và chính sách Cache

Ngoài Memory Cache và Disk Cache ở tầng thư viện tải ảnh, quá trình tải hình ảnh từ Internet còn liên quan đến các chính sách cache ở tầng mạng. Các thông tin

trong request và response có thể được sử dụng để xác định tài nguyên có thể được sử dụng lại hay cần được tải lại từ máy chủ.

Đối với Coil 3.x, Disk Cache được quản lý bởi thư viện và việc tuân theo các HTTP Cache-Control header không được kích hoạt mặc định. Nếu ứng dụng cần xử lý Cache-Control theo phản hồi của máy chủ, có thể cấu hình chiến lược cache tương ứng ở tầng network.

Điều này cho thấy việc có cache không đồng nghĩa với việc mọi hình ảnh luôn có thể được sử dụng ngoại tuyến. Khả năng hiển thị ảnh khi mất mạng còn phụ thuộc vào việc tài nguyên đó đã được tải và lưu trước đó, trạng thái cache và cấu hình tải hình ảnh của ứng dụng.

Trong thực nghiệm của đề tài, sự khác biệt giữa lần tải đầu tiên và lần tải lại, cũng như trường hợp có và không có kết nối mạng, được sử dụng để đánh giá hành vi của cơ chế cache.

Một vấn đề khác cần xem xét là tính hợp lệ của dữ liệu trong cache khi nội dung hình ảnh trên máy chủ thay đổi. Nếu cùng một URL được sử dụng nhưng nội dung tài nguyên đã được cập nhật, ứng dụng cần có cơ chế để tránh tiếp tục sử dụng dữ liệu cũ. Trong thực tế, vấn đề này có thể được xử lý thông qua chính sách cache phù hợp hoặc thay đổi phiên bản trong URL của hình ảnh để tạo một cache key mới.

Trong phạm vi đề tài, nhóm tập trung đánh giá khả năng tái sử dụng hình ảnh đã tải, lần tải đầu, lần tải lại và hoạt động khi mất kết nối mạng. Các trường hợp thay đổi nội dung ảnh trên máy chủ và cache invalidation chưa được thực nghiệm riêng biệt, do đó được xem là một giới hạn của quá trình đánh giá.

1.4 Bitmap, giải mã ảnh và quản lý bộ nhớ

1.4.1 Bitmap và giải mã hình ảnh

Các hình ảnh JPEG, PNG hoặc WebP thường được lưu hoặc truyền dưới dạng dữ liệu đã được nén. Trước khi hình ảnh có thể được hiển thị, dữ liệu này cần được giải mã (decode) thành biểu diễn hình ảnh phù hợp trong bộ nhớ.

Trên Android, Bitmap là một cấu trúc quan trọng dùng để biểu diễn dữ liệu pixel của hình ảnh. Kích thước file ảnh trên bộ nhớ lưu trữ và lượng RAM cần thiết sau khi giải mã là hai khái niệm khác nhau. Một file JPEG có dung lượng tương đối nhỏ vẫn có thể tạo ra một Bitmap lớn sau khi được giải mã nếu ảnh có độ phân giải cao.

Lượng bộ nhớ cơ bản cần thiết cho một Bitmap phụ thuộc vào chiều rộng, chiều cao và số byte sử dụng cho mỗi pixel:

Bitmap Memory = Width × Height × Bytes Per Pixel

Ví dụ, với cấu hình ARGB_8888, mỗi pixel thường cần 4 byte. Do đó, một hình ảnh có kích thước 1920 × 1080 pixel có thể cần khoảng:

1920 × 1080 × 4 ≈ 8,3 MB

bộ nhớ cho dữ liệu pixel sau khi được giải mã. Điều này giải thích vì sao việc tải đồng thời nhiều hình ảnh độ phân giải cao có thể làm mức sử dụng bộ nhớ tăng nhanh.

1.4.2 Định dạng hình ảnh

Ba định dạng thường gặp trong ứng dụng Android là JPEG, PNG và WebP.

JPEG phù hợp với hình ảnh có nhiều màu sắc và chi tiết như ảnh chụp. Định dạng này sử dụng nén mất dữ liệu (lossy compression), nhờ đó có thể giảm kích thước file nhưng chất lượng hình ảnh có thể giảm tùy theo mức nén.

PNG hỗ trợ nén không mất dữ liệu (lossless compression) và hỗ trợ độ trong suốt. PNG phù hợp với các loại hình ảnh như biểu tượng hoặc đồ họa cần giữ chi tiết, nhưng đối với ảnh chụp, dung lượng có thể lớn hơn JPEG.

WebP được thiết kế nhằm cung cấp khả năng nén hiệu quả và hỗ trợ nhiều trường hợp sử dụng khác nhau. Việc lựa chọn định dạng phù hợp có thể giúp giảm lượng dữ liệu cần tải, tuy nhiên hiệu năng thực tế còn phụ thuộc vào độ phân giải, kích thước file, nội dung ảnh và chi phí giải mã.

Do đó, không thể chỉ dựa vào kích thước file nén để đánh giá mức sử dụng bộ nhớ của một hình ảnh. Độ phân giải sau khi giải mã là một yếu tố quan trọng đối với mức tiêu thụ RAM.

1.4.3 Quản lý bộ nhớ

Bitmap có thể chiếm một phần đáng kể trong tổng bộ nhớ của ứng dụng. Khi nhiều Bitmap lớn được tạo ra cùng lúc, mức sử dụng bộ nhớ tăng và tạo ra memory pressure. Hệ thống Garbage Collector (GC) chịu trách nhiệm thu hồi các đối tượng không còn được tham chiếu, tuy nhiên việc tạo và giải phóng dữ liệu liên tục cũng có thể làm tăng chi phí quản lý bộ nhớ.

Trong trường hợp ứng dụng sử dụng nhiều bộ nhớ hơn giới hạn cho phép, nguy cơ xảy ra OutOfMemoryError (OOM) tăng lên. Vì vậy, ứng dụng không nên giải mã mọi hình ảnh ở độ phân giải gốc khi kích thước hiển thị thực tế nhỏ hơn đáng kể.

Đối với ứng dụng hiển thị danh sách nhiều hình ảnh, quản lý bộ nhớ cần được xem xét cùng với cache, kích thước ảnh và số lượng hình ảnh được xử lý tại một thời điểm thay vì đánh giá từng yếu tố riêng lẻ.

1.5 Lazy Loading và tối ưu hiển thị danh sách

Lazy Loading là kỹ thuật chỉ tạo hoặc tải nội dung khi nội dung đó thực sự cần thiết. Trong một danh sách gồm hàng trăm hoặc hàng nghìn phần tử, việc tạo và xử lý toàn bộ các phần tử ngay từ đầu có thể gây lãng phí tài nguyên.

Jetpack Compose cung cấp LazyColumn cho danh sách cuộn theo chiều dọc. Khác với việc tạo toàn bộ danh sách bằng Column, các lazy layout tập trung vào các item cần thiết trong vùng hiển thị. Điều này phù hợp với ứng dụng của đề tài khi số lượng phần tử được kiểm thử có thể tăng từ 100 lên 500 và 1000.

Lazy Loading đặc biệt quan trọng khi mỗi item chứa một hình ảnh từ Internet. Khi người dùng cuộn nhanh, các item liên tục xuất hiện và rời khỏi màn hình. Nếu tất cả yêu cầu tải ảnh đều tiếp tục được xử lý mặc dù kết quả không còn cần hiển thị, ứng dụng sẽ sử dụng thêm network, CPU và bộ nhớ một cách không cần thiết.

Sự kết hợp giữa LazyColumn và thư viện tải ảnh có khả năng quản lý vòng đời request giúp giảm lượng công việc không cần thiết. Đây cũng là cơ sở để đánh

giá độ mượt và tính chính xác của hình ảnh trong trường hợp người dùng cuộn nhanh qua danh sách.

1.6 Các kỹ thuật tối ưu hình ảnh

1.6.1 Resize và Downsampling

Một trong những vấn đề phổ biến khi xử lý hình ảnh là giải mã ảnh ở kích thước lớn hơn nhiều so với kích thước thực tế cần hiển thị. Ví dụ, nếu một hình ảnh có độ phân giải hàng nghìn pixel chỉ được sử dụng làm thumbnail kích thước nhỏ, việc giải mã toàn bộ ảnh gốc sẽ gây lãng phí bộ nhớ.

Resize là quá trình điều chỉnh hình ảnh về kích thước phù hợp với mục tiêu hiển thị. **Downsampling** giảm lượng dữ liệu pixel cần giải mã bằng cách lựa chọn kích thước đầu ra nhỏ hơn so với ảnh gốc.

Việc xác định kích thước ảnh gần với kích thước hiển thị giúp giảm lượng bộ nhớ dành cho Bitmap và có thể giảm chi phí giải mã. Các thư viện tải ảnh hiện đại như Coil có thể hỗ trợ downsampling khi kích thước đích của yêu cầu được xác định phù hợp.

1.6.2 Thumbnail

Thumbnail là phiên bản có kích thước nhỏ hơn của hình ảnh gốc. Đối với màn hình danh sách, người dùng thường không cần hình ảnh có độ phân giải tương đương với màn hình chi tiết.

Vì vậy, một chiến lược phù hợp là sử dụng hình ảnh có kích thước nhỏ hơn cho Product List và chỉ sử dụng kích thước lớn hơn khi người dùng mở Product Detail. Cách tiếp cận này giúp hạn chế lượng dữ liệu cần xử lý khi nhiều sản phẩm xuất hiện cùng lúc trên màn hình.

1.6.3 Placeholder và Error Handling

Quá trình tải ảnh từ Internet không phải lúc nào cũng hoàn tất ngay lập tức hoặc thành công. Tốc độ mạng có thể chậm, URL có thể không hợp lệ hoặc máy chủ có thể trả về lỗi.

Placeholder cung cấp nội dung tạm thời trong khi hình ảnh đang được tải. Nhờ đó, cấu trúc giao diện vẫn được duy trì thay vì xuất hiện một vùng trống không rõ trạng thái.

Khi yêu cầu tải ảnh thất bại, **error image** hoặc trạng thái lỗi giúp người dùng nhận biết rằng hình ảnh không thể được tải. Quan trọng hơn, lỗi của một yêu cầu ảnh không nên làm giao diện chính bị đóng băng hoặc khiến toàn bộ ứng dụng ngừng hoạt động.

1.6.4 Kết hợp các kỹ thuật tối ưu

Hiệu năng tải ảnh không phụ thuộc vào một kỹ thuật duy nhất. Cache có thể cải thiện tốc độ khi tải lại nhưng vẫn cần kiểm soát lượng RAM sử dụng. Resize giúp giảm kích thước Bitmap nhưng không thay thế được cache. Lazy Loading giảm số lượng item cần xử lý đồng thời nhưng hình ảnh của từng item vẫn cần được tải và giải mã phù hợp.

Do đó, một giải pháp tối ưu cần kết hợp nhiều kỹ thuật gồm **asynchronous loading, Memory Cache, Disk Cache, Lazy Loading, Resize/Downsampling, Thumbnail và xử lý trạng thái tải/lỗi**.

Đây cũng là cơ sở để đề tài xây dựng hai cấu hình **Baseline** và **Optimized** trên cùng tập dữ liệu và chức năng. Sự khác biệt giữa hai cấu hình sẽ được phân tích, cài đặt và kiểm chứng thông qua các chương tiếp theo.

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ ỨNG DỤNG

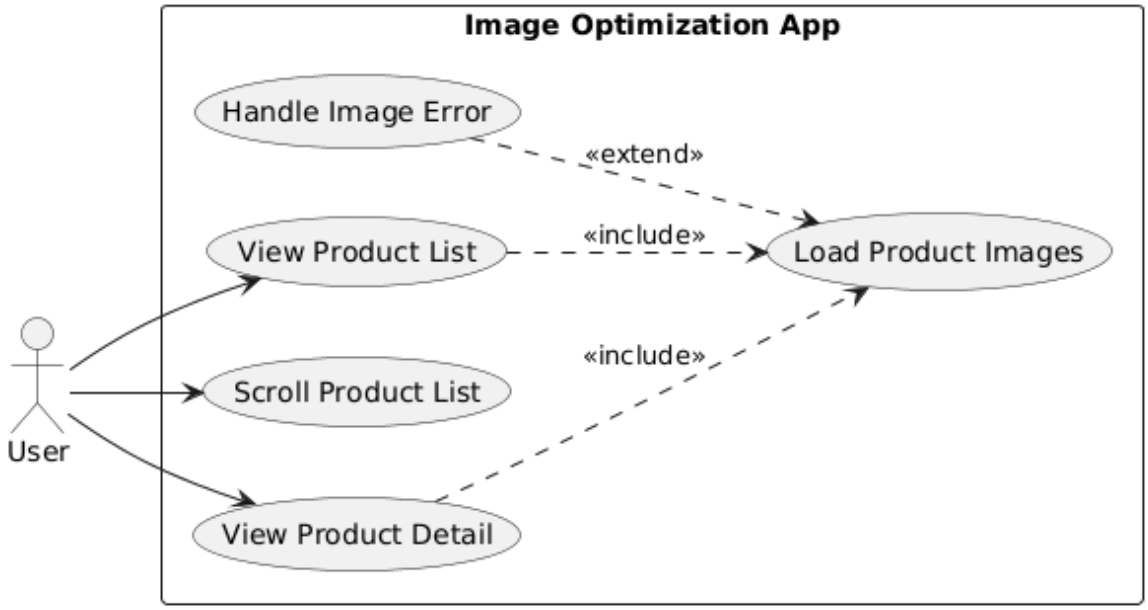
2.1 Phân tích yêu cầu ứng dụng

Ứng dụng được xây dựng nhằm phục vụ việc nghiên cứu và đánh giá hiệu năng tải, lưu trữ đệm và hiển thị hình ảnh trên Android. Hệ thống tập trung vào các chức năng cần thiết để tạo ra môi trường thực nghiệm có thể kiểm soát và so sánh giữa phiên bản cơ sở và phiên bản được tối ưu.

Các yêu cầu chính của ứng dụng bao gồm:

- Hiển thị danh sách tối thiểu 100 hình ảnh được tải từ URL.
- Hỗ trợ các tập dữ liệu có quy mô 100, 500 và 1000 phần tử.
- Cho phép người dùng cuộn danh sách và lựa chọn một sản phẩm để xem màn hình chi tiết.
- Tải hình ảnh bất đồng bộ và không thực hiện các tác vụ tải hoặc giải mã nặng trên Main Thread.
- Hiển thị placeholder trong thời gian tải và hình ảnh thay thế khi xảy ra lỗi.
- Hỗ trợ Memory Cache và Disk Cache.
- Hỗ trợ thay đổi kích thước ảnh phù hợp với vùng hiển thị.
- Đảm bảo hình ảnh được hiển thị đúng khi người dùng cuộn nhanh.
- Có hai cấu hình Baseline và Optimized để phục vụ quá trình so sánh hiệu năng.
- Hỗ trợ thực hiện các kịch bản kiểm thử như tải lần đầu, cache hit, mất mạng, URL lỗi, ảnh độ phân giải cao và danh sách nhiều phần tử.

Từ các yêu cầu trên, các chức năng chính mà người dùng trực tiếp tương tác gồm xem danh sách sản phẩm, cuộn danh sách, xem chi tiết sản phẩm và quan sát các trạng thái tải hoặc lỗi hình ảnh.



Hình 1. Sơ đồ Use Case của ứng dụng

2.2 Kiến trúc ứng dụng

Ứng dụng được phát triển bằng Kotlin và Jetpack Compose. Kiến trúc được thiết kế đơn giản nhằm tập trung vào quá trình tải và tối ưu hình ảnh thay vì các chức năng nghiệp vụ phức tạp.

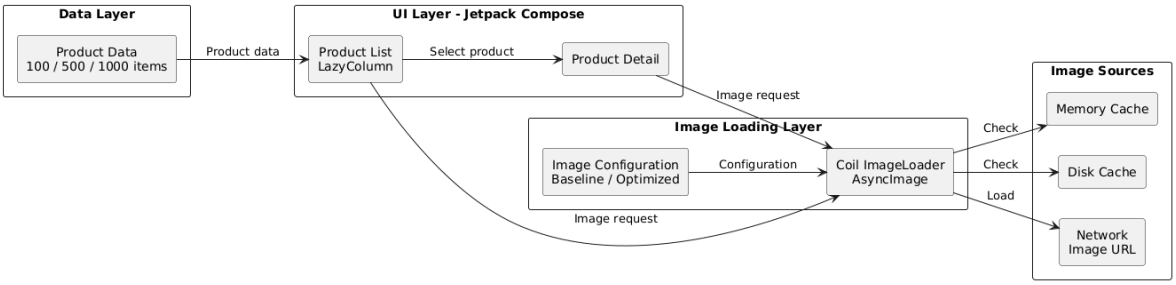
Hệ thống gồm các thành phần chính:

- **Data:** cung cấp dữ liệu sản phẩm và URL hình ảnh với các quy mô 100, 500 và 1000 phần tử.
- **User Interface:** gồm Product List và Product Detail được xây dựng bằng Jetpack Compose.
- **Image Loading:** sử dụng Coil và AsyncImage để thực hiện tải và hiển thị hình ảnh bất đồng bộ.
- **Cache:** quản lý việc sử dụng lại hình ảnh thông qua Memory Cache và Disk Cache.
- **Image Configuration:** quản lý hai chế độ Baseline và Optimized nhằm phục vụ quá trình thực nghiệm.

Luồng xử lý tổng quát có thể được mô tả như sau:

Data → UI → Image Request → Coil → Cache/Network → Decode/Resize → UI

Khi giao diện cần hiển thị một hình ảnh, yêu cầu được chuyển đến Coil. Hình ảnh có thể được lấy từ cache hoặc tải từ Internet tùy theo trạng thái dữ liệu. Sau khi được xử lý, kết quả được chuyển về giao diện để hiển thị.



Hình 2. Sơ đồ kiến trúc tổng quát của ứng dụng

2.3 Thiết kế luồng tải và lưu trữ đệm hình ảnh

Khi một hình ảnh cần được hiển thị, ứng dụng tạo image request dựa trên URL tương ứng. Trong thời gian chờ, placeholder được sử dụng để thể hiện trạng thái đang tải.

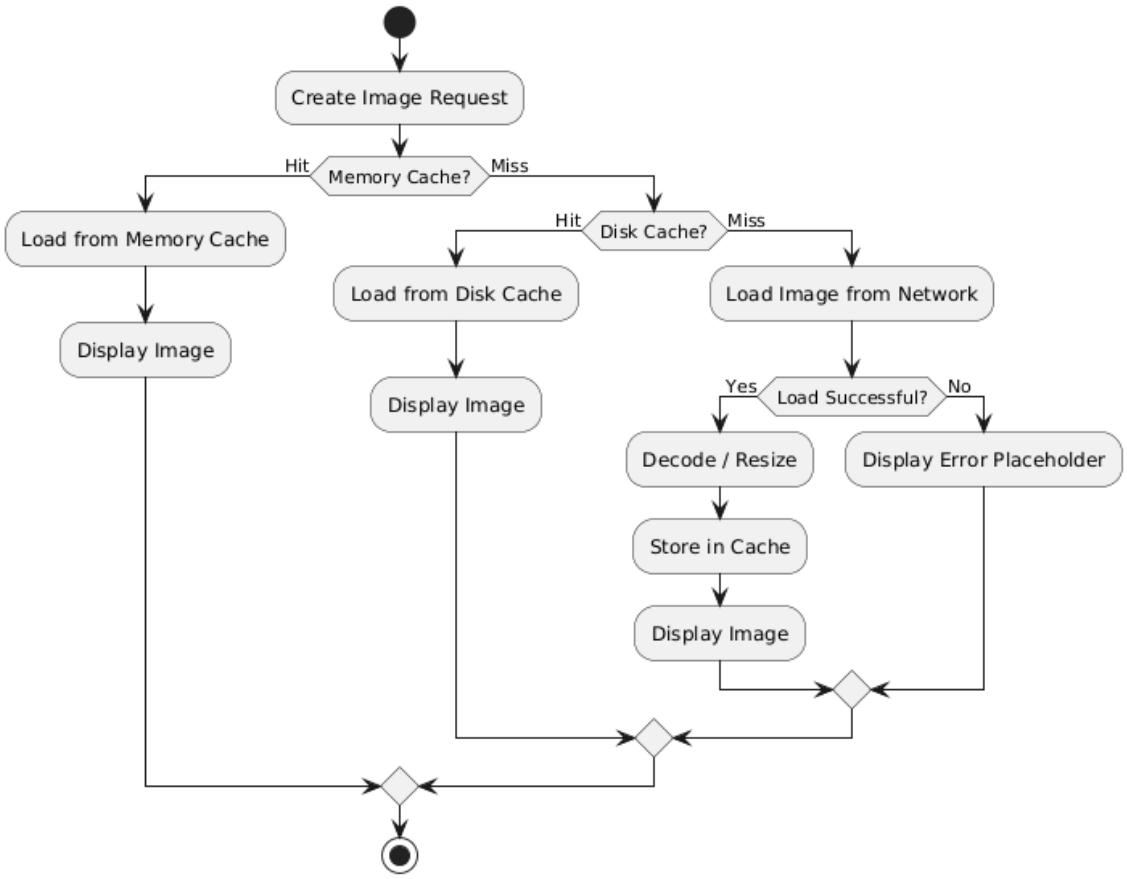
Quá trình xử lý được thiết kế theo luồng tổng quát:

Image Request → Memory Cache → Disk Cache → Network → Decode/Resize → Display

Đầu tiên, hệ thống kiểm tra khả năng sử dụng hình ảnh từ Memory Cache. Nếu không có dữ liệu phù hợp, Disk Cache được kiểm tra tiếp theo. Khi hình ảnh không thể được lấy từ cache, ứng dụng thực hiện yêu cầu mạng để tải dữ liệu.

Sau khi nhận được dữ liệu, hình ảnh được giải mã và xử lý theo cấu hình của phiên bản đang sử dụng trước khi hiển thị. Nếu yêu cầu thất bại do URL không hợp lệ, mất kết nối hoặc lỗi từ nguồn mạng, ứng dụng hiển thị error placeholder thay vì làm gián đoạn giao diện.

Đối với danh sách, LazyColumn được sử dụng để hạn chế việc tạo các thành phần giao diện không cần thiết. Khi người dùng cuộn, các image request được quản lý tương ứng với những item cần hiển thị.



Hình 3. Sơ đồ luồng tải và lưu trữ đệm hình ảnh

2.4 Thiết kế Baseline và Optimized

Để đánh giá tác động của các kỹ thuật tối ưu, ứng dụng được thiết kế với hai chế độ **Baseline** và **Optimized**. Cả hai sử dụng cùng tập dữ liệu, giao diện và chức năng chính để đảm bảo điều kiện so sánh tương đối thống nhất.

Baseline được sử dụng làm phiên bản tham chiếu, với cách tải và hiển thị hình ảnh ở mức cơ bản. Phiên bản này giúp ghi nhận trạng thái hiệu năng trước khi áp dụng các cấu hình tối ưu chủ động.

Optimized áp dụng các kỹ thuật như sử dụng cache phù hợp, resize/downsampling hình ảnh theo nhu cầu hiển thị và hạn chế xử lý hình ảnh có kích thước lớn không cần thiết.

Việc sử dụng cùng chức năng và dữ liệu giúp giảm ảnh hưởng của các yếu tố không liên quan, từ đó hỗ trợ việc đánh giá sự khác biệt về hiệu năng giữa hai phiên bản.

2.5 Thiết kế kịch bản thực nghiệm

Các kịch bản thực nghiệm được xây dựng để đánh giá ứng dụng trong nhiều điều kiện sử dụng khác nhau. Nội dung kiểm thử tập trung vào thời gian tải và hiển thị hình ảnh, hiệu quả của cache, khả năng cuộn nhanh, mức sử dụng bộ nhớ, xử lý khi mất mạng hoặc URL lỗi và khả năng hoạt động khi số lượng phần tử tăng.

Từ đó, đề tài xây dựng tám trường hợp kiểm thử từ **TC01 đến TC08**, bao gồm tải lần đầu, cache hit, cuộn nhanh, hình ảnh độ phân giải cao, offline, lỗi mạng, thay đổi quy mô dữ liệu và so sánh tổng thể giữa Baseline với Optimized.

Các test case được thực hiện trong điều kiện tương đương giữa hai phiên bản nhằm tạo cơ sở cho việc phân tích hiệu năng. Quy trình thực nghiệm và nội dung chi tiết của từng test case được trình bày tại Chương 4.

CHƯƠNG 3. CÀI ĐẶT VÀ TỐI ƯU ỨNG DỤNG

3.1 Môi trường và công nghệ phát triển

Ứng dụng được phát triển trên nền tảng Android bằng ngôn ngữ Kotlin và sử dụng Jetpack Compose để xây dựng giao diện. Android Studio được sử dụng làm môi trường phát triển chính, hỗ trợ viết mã nguồn, quản lý thư viện, build, chạy và kiểm thử ứng dụng.

Các công nghệ chính được sử dụng trong đề tài gồm:

- **Kotlin:** ngôn ngữ lập trình chính.
- **Jetpack Compose:** xây dựng giao diện ứng dụng.
- **LazyColumn:** hiển thị danh sách có số lượng phần tử lớn.
- **Coil:** tải và hiển thị hình ảnh từ URL.
- **Android Studio Profiler:** theo dõi CPU và bộ nhớ trong quá trình thực nghiệm.
- **Git và GitHub:** quản lý phiên bản và lưu trữ mã nguồn.

Ứng dụng được cấu hình với minSdk 24, compileSdk 37 và targetSdk 37. Trong quá trình kiểm thử, nhóm sử dụng thiết bị giả lập Pixel 5 để duy trì môi trường tương đối thống nhất giữa các lần thực nghiệm.

3.2 Cài đặt các chức năng của ứng dụng

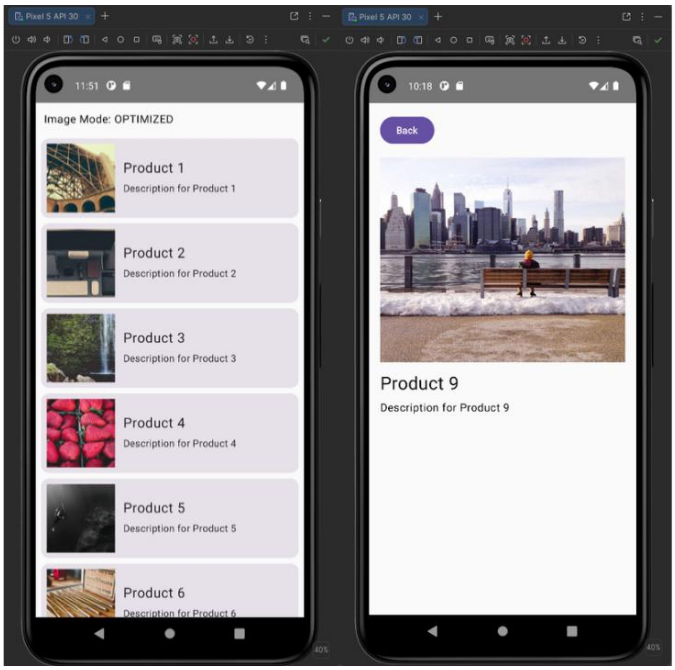
Ứng dụng được xây dựng với các chức năng chính gồm hiển thị danh sách hình ảnh, xem chi tiết, xử lý trạng thái tải, xử lý lỗi ảnh và hỗ trợ cuộn danh sách. Đây cũng là các chức năng cơ sở được duy trì trong cả hai phiên bản Baseline và Optimized.

3.2.1 Danh sách và màn hình chi tiết

Màn hình Product List sử dụng LazyColumn để hiển thị danh sách sản phẩm. Dữ liệu được thiết kế để có thể thay đổi giữa ba quy mô **100, 500 và 1000 phần tử**, phục vụ cho các phép thử hiệu năng ở các chương sau.

Mỗi phần tử trong danh sách hiển thị thông tin cơ bản và hình ảnh tương ứng được tải từ URL. Khi người dùng chọn một sản phẩm, ứng dụng chuyển sang Product Detail để hiển thị hình ảnh ở kích thước lớn hơn.

Việc xây dựng hai màn hình này cho phép đề tài đánh giá cả trường hợp hiển thị nhiều hình ảnh trong danh sách và trường hợp hiển thị một hình ảnh lớn ở màn hình chi tiết.



Hình 4. Giao diện Product List và Product Detail của ứng dụng

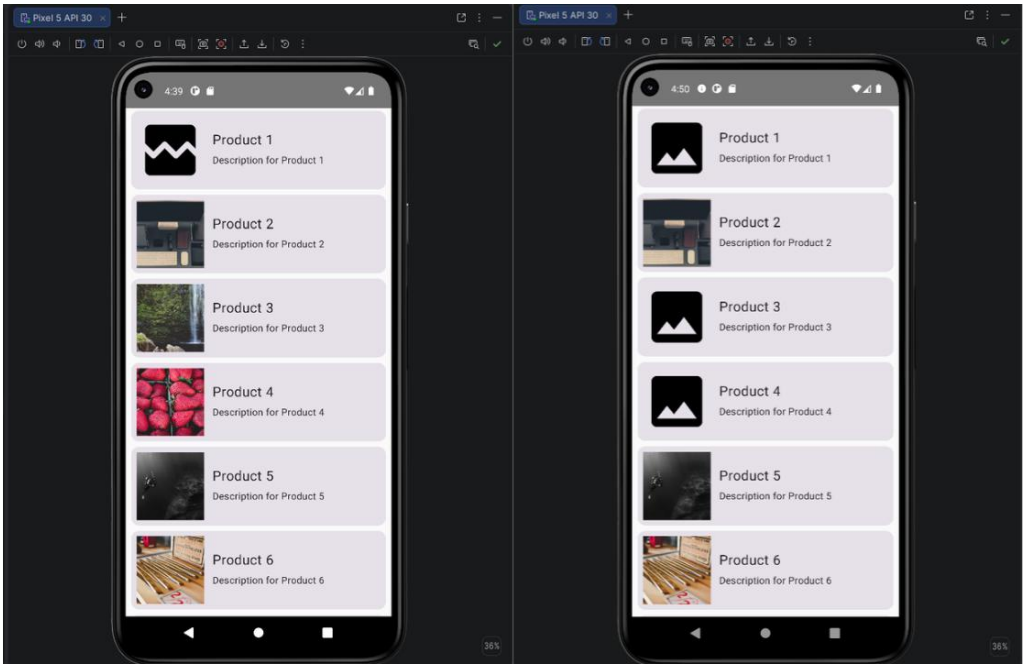
3.2.2 Trạng thái tải, lỗi ảnh và xử lý cuộn

Trong quá trình tải hình ảnh, ứng dụng sử dụng placeholder để thể hiện trạng thái đang chờ dữ liệu. Khi yêu cầu tải thất bại do URL không hợp lệ hoặc lỗi kết nối, error image được hiển thị thay cho hình ảnh. Hình 3.2 minh họa hai trạng thái này trong quá trình chạy thực tế của ứng dụng.

Cách xử lý này giúp duy trì bố cục giao diện và tránh trường hợp lỗi của một hình ảnh ảnh hưởng đến toàn bộ danh sách.

Đối với thao tác cuộn, LazyColumn được sử dụng để quản lý các phần tử trong danh sách. Khi người dùng cuộn nhanh, các item mới liên tục xuất hiện và các yêu

cầu tải ảnh được cập nhật tương ứng. Đây là cơ sở để kiểm tra khả năng hiển thị đúng ảnh, độ phản hồi của giao diện và mức sử dụng tài nguyên khi danh sách có số lượng phần tử lớn.



Hình 5. Trạng thái Loading và Error trong quá trình tải hình ảnh

3.3 Cài đặt mô-đun tải ảnh và Cache

Ứng dụng sử dụng thư viện Coil để thực hiện quá trình tải và hiển thị hình ảnh từ Internet. Trong giao diện Jetpack Compose, AsyncImage được sử dụng để liên kết URL hình ảnh với thành phần hiển thị. Coil hỗ trợ thực hiện image request bất đồng bộ, nhờ đó quá trình tải ảnh không làm chặn hoạt động chính của giao diện. Ứng dụng đồng thời cấu hình placeholder và error image để xử lý các trạng thái khác nhau của quá trình tải.

Trong ứng dụng, ImageRequest được triển khai theo hai cấu hình tương ứng với hai chế độ thực nghiệm. Ở chế độ Baseline, ImageRequest sử dụng các thiết lập cơ bản và chủ yếu dựa trên cấu hình mặc định của Coil. Vì vậy, Baseline vẫn có thể sử dụng các cơ chế cache mặc định của thư viện và không được xem là phiên bản hoàn toàn không có cache.

Ở chế độ Optimized, ImageRequest được cấu hình tường minh với size(width, height), memoryCachePolicy(CachePolicy.ENABLED) và diskCachePolicy(CachePolicy.ENABLED). Việc xác định kích thước ảnh giúp nhóm chủ động kiểm soát kích thước yêu cầu tải, trong khi các cache policy thể hiện rõ chính sách sử dụng Memory Cache và Disk Cache trong cấu hình Optimized.

Trong giao diện Product List, ImageRequest được truyền vào AsyncImage cùng với placeholder, error image và ContentScale.Crop. Hình 3.3 minh họa phần cài đặt ImageRequest của chế độ Optimized và cách request này được sử dụng để hiển thị hình ảnh trong giao diện.

```
fun createOptimizedRequest(
    context: Context,
    imageUrl: String,
    width: Int,
    height: Int
): ImageRequest {
    return ImageRequest.Builder(context)
        .data(imageUrl)
        .size(width, height)
        .memoryCachePolicy(CachePolicy.ENABLED)
        .diskCachePolicy(CachePolicy.ENABLED)
        .build()
}
```

```
AsyncImage(
    model = createImageRequest(
        context = LocalContext.current,
        imageUrl = product.imageUrl,
        width = 200,
        height = 200
    ),
    contentDescription = product.name,
    placeholder = painterResource( id = R.drawable.placeholder_image),
    error = painterResource( id = R.drawable.error_image),
    modifier = Modifier.size(100.dp),
    contentScale = ContentScale.Crop
)
```

Hình 6. Cấu hình ImageRequest và hiển thị hình ảnh bằng AsyncImage trong chế độ Optimized

Cần lưu ý rằng AsyncImage cũng có khả năng xác định kích thước tải dựa trên vùng hiển thị của thành phần giao diện. Do đó, việc sử dụng size(width, height) trong cấu hình Optimized được xem là cách chủ động xác định kích thước yêu cầu tải ảnh, thay vì khẳng định rằng Baseline luôn tải và giải mã hình ảnh ở kích thước nguyên bản.

Các cấu hình trên được sử dụng làm cơ sở cho những phép thử về lần tải đầu tiên, cache hit, ảnh độ phân giải cao, chế độ offline và mức sử dụng bộ nhớ được trình bày trong các chương tiếp theo.

3.4 Xây dựng hai phiên bản Baseline và Optimized

Để đánh giá ảnh hưởng của cấu hình xử lý hình ảnh, ứng dụng được xây dựng với hai chế độ là Baseline và Optimized. Hai chế độ sử dụng cùng tập dữ liệu, giao diện và các chức năng chính nhằm đảm bảo điều kiện thực nghiệm có thể được so sánh.

Baseline được sử dụng làm mốc tham chiếu. Ở chế độ này, ImageRequest được tạo với các thiết lập cơ bản và chủ yếu sử dụng cấu hình mặc định của Coil. Ứng dụng vẫn đảm bảo đầy đủ các chức năng như hiển thị danh sách sản phẩm, xem chi tiết, tải ảnh bất đồng bộ, placeholder và xử lý lỗi.

Optimized sử dụng cấu hình ImageRequest tường minh hơn. Kích thước yêu cầu tải ảnh được xác định thông qua size(width, height), đồng thời Memory Cache và Disk Cache policy được khai báo ở trạng thái ENABLED. Cách cấu hình này giúp nhóm chủ động kiểm soát ImageRequest trong quá trình thực nghiệm.

Nội dung	Baseline	Optimized
Dataset	100 / 500 / 1000	100 / 500 / 1000
Product List	Có	Có
Product Detail	Có	Có
Async Image Loading	Có	Có
Placeholder / Error	Có	Có
LazyColumn	Có	Có
Image Cache	Theo cấu hình mặc định của Coil	Cache policy được khai báo tường minh

Resize / Downsampling	Theo cơ chế xử lý của AsyncImage/Coil	Chủ động cấu hình size(width, height)
Mục đích	Mốc tham chiếu	Cấu hình sau tối ưu

Việc chuyển đổi giữa hai chế độ được quản lý thông qua cấu hình trong mã nguồn. Nhờ đó, nhóm có thể thực hiện cùng một kịch bản kiểm thử trên Baseline và Optimized mà không cần xây dựng hai ứng dụng có chức năng khác nhau.

Hình 3.4 minh họa sự khác biệt trong cách tạo ImageRequest giữa hai chế độ và cách ứng dụng lựa chọn cấu hình tương ứng trong quá trình chạy.

```
fun createBaselineRequest(
    context: Context,
    imageUrl: String
): ImageRequest {
    return ImageRequest.Builder(context)
        .data(imageUrl)
        .build()
}

1 Usage
fun createOptimizedRequest(
    context: Context,
    imageUrl: String,
    width: Int,
    height: Int
): ImageRequest {
    return ImageRequest.Builder(context)
        .data(imageUrl)
        .size(width, height)
        .memoryCachePolicy(CachePolicy.ENABLED)
        .diskCachePolicy(CachePolicy.ENABLED)
        .build()
}
```

```
fun createImageRequest(
    context: Context,
    imageUrl: String,
    width: Int,
    height: Int
): ImageRequest {
    return when (ImageConfig.currentMode) {
        ImageMode.BASELINE -> {
            createBaselineRequest(
                context = context,
                imageUrl = imageUrl
            )
        }

        ImageMode.OPTIMIZED -> {
            createOptimizedRequest(
                context = context,
                imageUrl = imageUrl,
                width = width,
                height = height
            )
        }
    }
}
```

Hình 7. Cài đặt và lựa chọn cấu hình ImageRequest giữa Baseline và Optimized

3.5 Quản lý mã nguồn và hướng dẫn Build

Mã nguồn của ứng dụng được quản lý bằng Git và lưu trữ trên GitHub. Repository bao gồm mã nguồn Kotlin, tài nguyên ứng dụng, cấu hình Gradle và các tài liệu cần thiết để chạy lại project.

Quy trình cài đặt và chạy ứng dụng được thực hiện theo các bước cơ bản:

Clone Repository → Open Project → Gradle Sync → Select Device → Build & Run

Đầu tiên, repository được clone về máy tính và mở bằng Android Studio. Sau khi project được mở, Gradle Sync được thực hiện để đồng bộ cấu hình và tải các dependency cần thiết. Người dùng sau đó lựa chọn thiết bị Android hoặc máy ảo phù hợp và sử dụng chức năng Run của Android Studio để build, cài đặt và khởi chạy ứng dụng.

Tệp README.md trong repository được sử dụng để mô tả yêu cầu môi trường, cách build và chạy ứng dụng, cấu trúc project và các thông tin cần thiết để tái hiện quá trình thực nghiệm.

Repository: *https://github.com/ThanhTien35/ImageOptimizationApp*

Việc sử dụng Git giúp quản lý các thay đổi trong quá trình phát triển và lưu lại các phiên bản của ứng dụng, đồng thời đáp ứng yêu cầu về mã nguồn và khả năng tái tạo sản phẩm.

3.6 Kết quả cài đặt

Sau quá trình phát triển, ứng dụng đã hoàn thiện các chức năng cần thiết gồm hiển thị danh sách hình ảnh, màn hình chi tiết, xử lý trạng thái loading/error, cuộn danh sách và tải hình ảnh từ Internet. Mô-đun tải ảnh sử dụng Coil hỗ trợ cache và các cấu hình xử lý hình ảnh phục vụ quá trình tối ưu.

Hai chế độ Baseline và Optimized được xây dựng trên cùng ứng dụng để phục vụ quá trình so sánh. Trên cơ sở sản phẩm đã hoàn thiện, các kịch bản thực nghiệm được thiết kế và thực hiện nhằm đánh giá sự khác biệt về hiệu năng giữa hai chế độ. Nội dung này được trình bày trong các chương tiếp theo.

CHƯƠNG 4. PHƯƠNG PHÁP VÀ KỊCH BẢN THỰC NGHIỆM

4.1 Môi trường và công cụ thực nghiệm

Sau khi hoàn thiện hai chế độ Baseline và Optimized, nhóm tiến hành thực nghiệm nhằm đánh giá hoạt động của ứng dụng trong các tình huống tải và hiển thị hình ảnh khác nhau. Các phép thử được thực hiện trên cùng môi trường để hạn chế sự khác biệt do thiết bị và cấu hình hệ thống.

Môi trường thực nghiệm chính gồm:

- **Thiết bị giả lập:** Pixel 5.
- **Hệ điều hành:** Android API 30.
- **Môi trường phát triển:** Android Studio.
- **Ngôn ngữ:** Kotlin.
- **Giao diện:** Jetpack Compose.
- **Thư viện tải ảnh:** Coil.
- **Công cụ theo dõi hiệu năng:** Android Studio Profiler.
- **Quy mô dữ liệu:** 100, 500 và 1000 phần tử.

Android Studio Profiler được sử dụng để quan sát mức sử dụng bộ nhớ và hoạt động của ứng dụng trong quá trình thực nghiệm. Bên cạnh số liệu từ Profiler, nhóm sử dụng ảnh chụp màn hình và kết quả hiển thị thực tế để kiểm tra các trường hợp như cache hit, mất kết nối mạng, URL lỗi và cuộn nhanh.

Hai chế độ Baseline và Optimized sử dụng cùng dữ liệu, chức năng và môi trường chạy. Điểm khác biệt chủ yếu nằm ở cấu hình xử lý hình ảnh đã trình bày tại Chương 3. Cách thiết lập này giúp việc so sánh tập trung vào ảnh hưởng của các kỹ thuật tối ưu thay vì sự khác biệt về chức năng của ứng dụng.

4.2 Các chỉ số và tiêu chí đánh giá

Quá trình thực nghiệm tập trung vào các tiêu chí có liên quan trực tiếp đến hiệu năng tải và hiển thị hình ảnh, bao gồm khả năng tải ảnh, sử dụng cache, mức sử dụng bộ nhớ, khả năng phản hồi khi cuộn và xử lý lỗi.

Khả năng tải và hiển thị hình ảnh: kiểm tra hình ảnh có được tải và hiển thị đúng trong lần truy cập đầu tiên, khi truy cập lại và trong các điều kiện mạng khác nhau hay không. Quá trình này được sử dụng để đánh giá hành vi của ứng dụng trong các trường hợp first load và cache hit.

Khả năng sử dụng Cache: đánh giá khả năng tái sử dụng những hình ảnh đã được tải trước đó. Baseline chủ yếu sử dụng cấu hình mặc định của Coil, trong khi Optimized khai báo tường minh Memory Cache và Disk Cache policy trong ImageRequest. Việc kiểm thử được thực hiện thông qua lần tải đầu, lần truy cập lại và trường hợp thiết bị mất kết nối mạng.

Mức sử dụng bộ nhớ: theo dõi lượng bộ nhớ mà ứng dụng sử dụng trong các tình huống khác nhau, đặc biệt khi hiển thị hình ảnh có kích thước lớn hoặc khi tăng số lượng phần tử trong danh sách. Android Studio Profiler được sử dụng để ghi nhận các giá trị bộ nhớ tại thời điểm kiểm thử và so sánh giữa Baseline và Optimized.

Thời gian phản hồi và độ mượt: quan sát quá trình tải, hiển thị và cuộn danh sách nhằm kiểm tra tình trạng tải chậm, hiển thị sai ảnh, phản hồi không ổn định hoặc giật trong quá trình sử dụng. Trong phạm vi thực nghiệm hiện tại, các tiêu chí này chủ yếu được đánh giá thông qua hành vi thực tế và ảnh chụp minh chứng; nhóm chưa sử dụng benchmark tự động để thu thập thời gian tải, thời gian giải mã hoặc số lượng khung hình bị giật một cách định lượng.

Khả năng xử lý lỗi: kiểm tra phản ứng của ứng dụng khi URL hình ảnh không hợp lệ, quá trình tải thất bại, máy chủ phản hồi chậm hoặc thiết bị không có kết nối mạng. Ứng dụng cần duy trì giao diện ổn định và hiển thị Loading/Error placeholder phù hợp.

Các tiêu chí trên được kết hợp để đánh giá ứng dụng thay vì chỉ dựa trên một giá trị riêng lẻ. Trong đó, số liệu Memory được sử dụng cho so sánh định lượng, còn

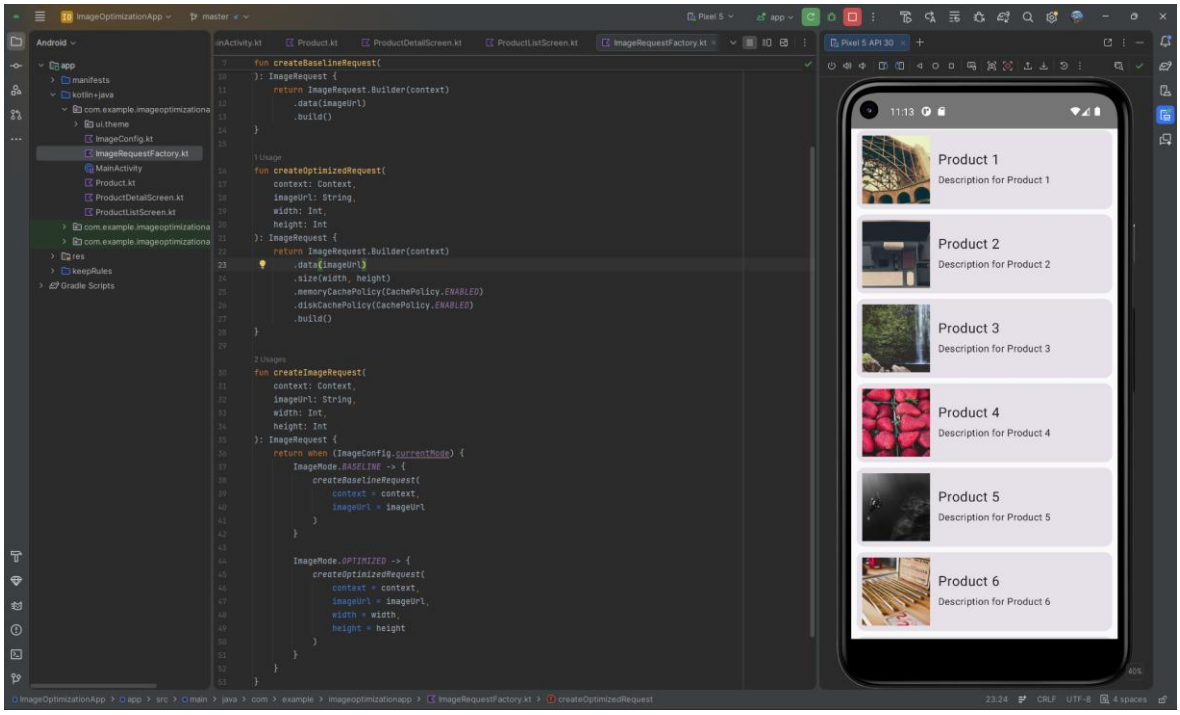
các kết quả về cache, xử lý lỗi và khả năng phản hồi được sử dụng để đánh giá hành vi của ứng dụng trong các kịch bản kiểm thử tương ứng.

4.3 Quy trình thực nghiệm

Mỗi kịch bản được thực hiện trên phiên bản phù hợp hoặc trên cả hai chế độ Baseline và Optimized tùy theo mục tiêu của phép thử. Quy trình thực nghiệm tổng quát gồm các bước:

1. Chọn chế độ **Baseline** hoặc **Optimized** trong cấu hình ứng dụng.
2. Thiết lập số lượng phần tử cần kiểm thử, chẳng hạn 100, 500 hoặc 1000 sản phẩm.
3. Build và chạy ứng dụng trên cùng thiết bị giả lập.
4. Thực hiện thao tác tương ứng với từng test case.
5. Theo dõi giao diện và Android Studio Profiler trong quá trình chạy.
6. Ghi nhận kết quả, số liệu hoặc ảnh chụp làm minh chứng.
7. Lặp lại kịch bản trên chế độ còn lại khi cần so sánh.
8. Tổng hợp kết quả để phân tích sự khác biệt giữa Baseline và Optimized.

Đối với những phép thử liên quan đến cache và mạng, trạng thái trước khi chạy được kiểm soát phù hợp với mục tiêu của test case. Ví dụ, lần tải đầu cần phân biệt với lần mở lại đã có dữ liệu cache; trong phép thử offline, chỉ những hình ảnh đã được tải và lưu trữ trước đó mới có khả năng được sử dụng lại khi thiết bị mất kết nối mạng.



Hình 8. Môi trường thực nghiệm trên Android Studio và thiết bị giả lập Pixel 5 API 30

Môi trường thực nghiệm được duy trì thống nhất trong quá trình kiểm thử nhằm hạn chế ảnh hưởng của sự khác biệt về thiết bị và cấu hình. Hình 4.1 minh họa ứng dụng được chạy trực tiếp trên thiết bị giả lập Pixel 5 API 30 trong Android Studio.

4.4 Các trường hợp kiểm thử

Bộ kiểm thử được xây dựng gồm tám trường hợp, bao phủ các tình huống chính liên quan đến lần tải đầu tiên, khả năng sử dụng cache, cuộn nhanh, hiển thị hình ảnh kích thước lớn, hoạt động khi mất kết nối mạng, xử lý lỗi và thay đổi quy mô dữ liệu. Mỗi trường hợp được thực hiện trong cùng môi trường thực nghiệm nhằm duy trì điều kiện so sánh giữa Baseline và Optimized.

Test case	Kịch bản	Mục tiêu kiểm thử	Tiêu chí đánh giá
TC01	First Load	Kiểm tra lần tải danh sách đầu tiên và ghi nhận trạng thái hiển thị,	Ảnh được tải và hiển thị đúng; placeholder hoạt động; ứng

		khả năng phản hồi và mức sử dụng bộ nhớ.	dụng không xảy ra lỗi hoặc crash.
TC02	Cache Hit	Mở lại nội dung đã tải để kiểm tra khả năng tái sử dụng dữ liệu từ cache.	Ảnh đã tải trước đó có khả năng được tái sử dụng trong điều kiện cache phù hợp; giao diện hoạt động ổn định.
TC03	Fast Scroll	Cuộn nhanh danh sách để đánh giá khả năng phản hồi và hiển thị hình ảnh.	Danh sách tiếp tục phản hồi, ảnh tương ứng với item và không xảy ra crash trong quá trình cuộn.
TC04	High Resolution	Hiển thị ảnh kích thước lớn ở màn hình chi tiết và theo dõi ảnh hưởng đến bộ nhớ.	Ảnh được hiển thị đúng; ứng dụng hoạt động ổn định; mức Memory được ghi nhận để so sánh hai cấu hình.
TC05	Offline	Ngắt kết nối mạng sau khi đã tải dữ liệu để kiểm tra khả năng sử dụng ảnh đã được cache.	Ảnh đã có dữ liệu cache có thể được tái sử dụng trong điều kiện phù hợp; ảnh chưa được tải trước đó không được giả định là có sẵn khi offline.
TC06	Error / Slow Network	Kiểm tra URL lỗi hoặc điều kiện tải không thuận lợi và khả năng xử lý trạng thái lỗi.	Loading/Error placeholder hoạt động đúng; bố cục giao diện được duy trì và ứng dụng không crash.
TC07	Dataset Scaling	Thực nghiệm với 100, 500 và 1000 phần tử để đánh giá ảnh hưởng của quy mô dữ liệu.	Danh sách tiếp tục hoạt động ở các quy mô dữ liệu; Memory được ghi nhận để so sánh Baseline và Optimized.

TC08	Before / After Comparison	Tổng hợp kết quả giữa Baseline và Optimized.	Đối chiếu các kết quả TC01–TC07 để đánh giá tổng thể hai cấu hình.

4.4.1 TC01 – First Load và TC02 – Cache Hit

TC01 được sử dụng để đánh giá hành vi của ứng dụng trong lần tải đầu tiên, khi hình ảnh chưa được tái sử dụng từ cache của lần tải trước. Nhóm quan sát quá trình hiển thị placeholder, hình ảnh sau khi tải và mức sử dụng bộ nhớ của ứng dụng thông qua Android Studio Profiler.

Sau đó, TC02 thực hiện truy cập lại nội dung đã được tải nhằm kiểm tra khả năng tái sử dụng dữ liệu từ cache. Hai kịch bản hỗ trợ đối chiếu hành vi của ứng dụng giữa lần tải ban đầu và trường hợp dữ liệu hình ảnh đã được tải trước đó.

Trong phạm vi thực nghiệm hiện tại, TC02 tập trung vào hành vi cache hit đối với dữ liệu đã được tải. Trường hợp xóa hoàn toàn cache và khởi động lại ứng dụng chưa được tách thành một phép thử độc lập.

4.4.2 TC03 – Fast Scroll và TC04 – High Resolution

Trong TC03, danh sách được cuộn nhanh qua nhiều phần tử liên tiếp. Mục tiêu là kiểm tra khả năng cập nhật hình ảnh tương ứng với từng item và quan sát độ ổn định của giao diện khi nhiều yêu cầu hình ảnh xuất hiện trong thời gian ngắn. Độ mượt trong trường hợp này được đánh giá thông qua hành vi thực tế của ứng dụng; nhóm chưa thu thập số lượng jank frame hoặc FPS bằng benchmark định lượng.

TC04 tập trung vào màn hình Product Detail, nơi hình ảnh được hiển thị với kích thước lớn hơn so với Product List. Mức sử dụng bộ nhớ được theo dõi bằng Android Studio Profiler nhằm đánh giá ảnh hưởng của quá trình xử lý hình ảnh và so sánh kết quả giữa Baseline và Optimized.

4.4.3 TC05 – Offline và TC06 – Error/Slow Network

TC05 được thực hiện bằng cách tải trước một số hình ảnh, sau đó ngắt kết nối mạng và truy cập lại nội dung. Kịch bản này nhằm kiểm tra khả năng tái sử dụng

những hình ảnh đã được lưu trong cache. Các hình ảnh chưa từng được tải trước đó không được xem là dữ liệu có sẵn khi thiết bị offline.

TC06 kiểm tra khả năng xử lý khi quá trình tải hình ảnh thất bại hoặc gặp điều kiện mạng không thuận lợi. Trong trường hợp URL không hợp lệ hoặc request thất bại, ứng dụng cần hiển thị Error placeholder và duy trì bố cục giao diện thay vì gây lỗi toàn bộ màn hình.

4.4.4 TC07 – Dataset Scaling và TC08 – Before/After Comparison

TC07 thay đổi số lượng phần tử lần lượt ở các mức 100, 500 và 1000 sản phẩm. Qua đó, nhóm theo dõi mức sử dụng bộ nhớ và khả năng duy trì hoạt động của danh sách khi quy mô dữ liệu tăng. Các giá trị Memory được ghi nhận tại thời điểm kiểm thử và được sử dụng để so sánh giữa Baseline và Optimized.

TC08 không được xem là một phép thử độc lập mà là bước tổng hợp kết quả của TC01–TC07. Các số liệu Profiler, hành vi cache, khả năng xử lý lỗi và hoạt động của giao diện được đối chiếu giữa Baseline và Optimized để đánh giá tổng thể tác động của cấu hình tối ưu. Kết quả cụ thể và các nhận xét được trình bày trong Chương 5.

CHƯƠNG 5. KẾT QUẢ VÀ PHÂN TÍCH HIỆU NĂNG

5.1 Kết quả các trường hợp kiểm thử

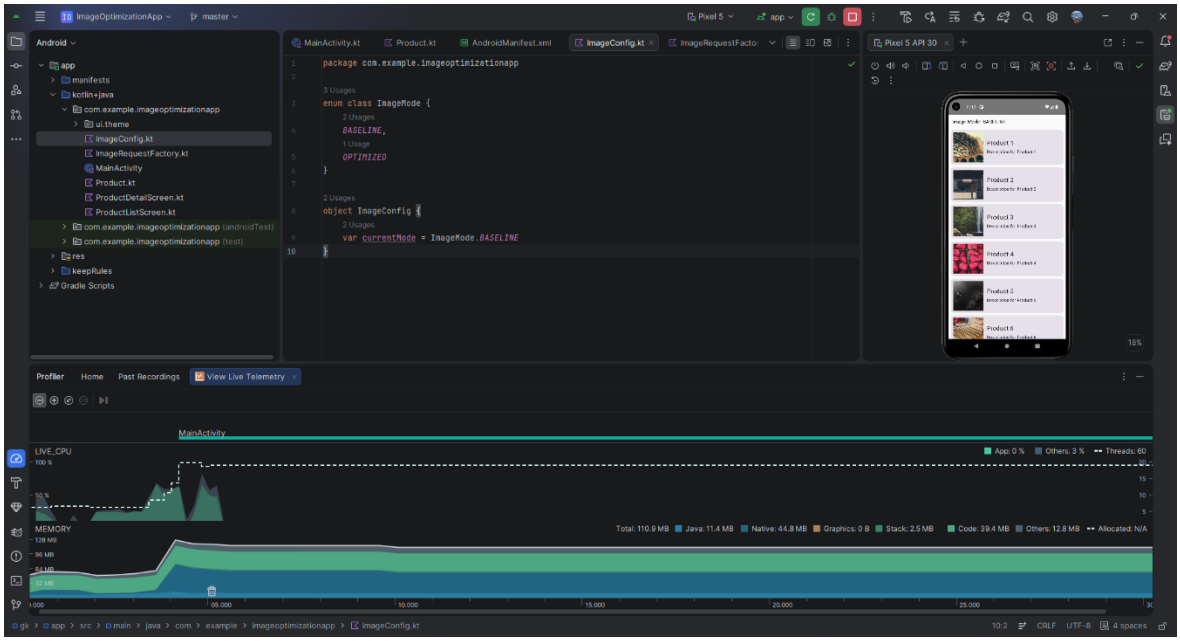
Dựa trên phương pháp thực nghiệm được trình bày ở Chương 4, ứng dụng được kiểm thử trên hai cấu hình Baseline và Optimized trong cùng môi trường. Các trường hợp kiểm thử tập trung vào quá trình tải hình ảnh lần đầu, khả năng sử dụng bộ nhớ đệm, thao tác cuộn nhanh, hiển thị hình ảnh độ phân giải cao, hoạt động khi mất kết nối mạng, xử lý URL lỗi và khả năng làm việc với số lượng dữ liệu khác nhau.

5.1.1 TC01 – First Load

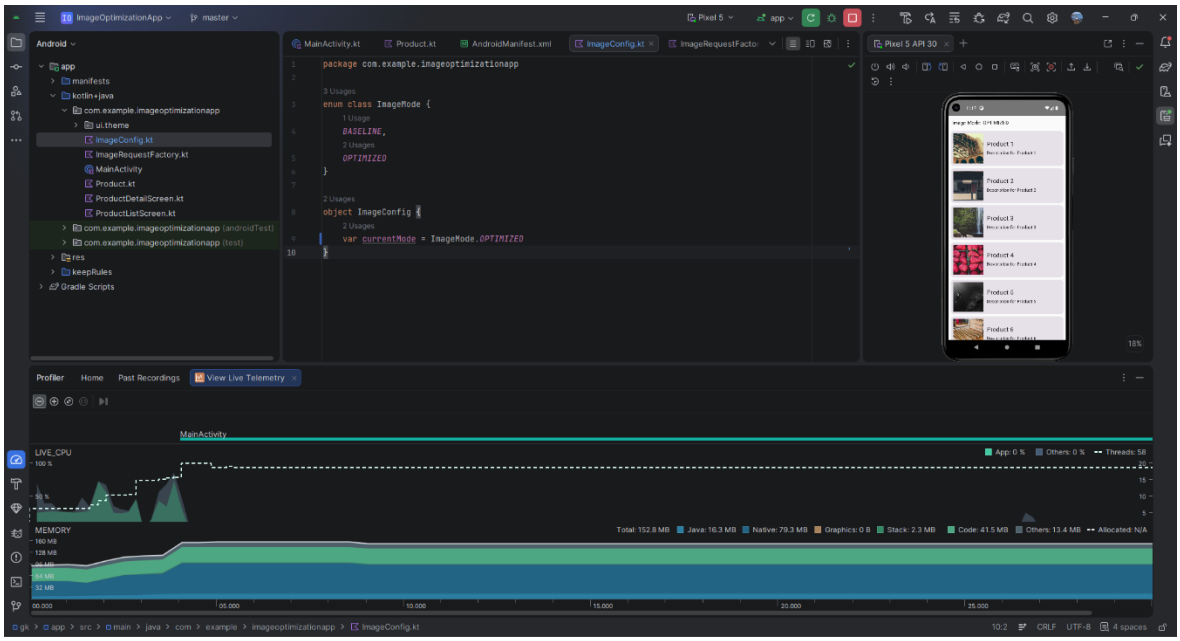
TC01 đánh giá quá trình tải hình ảnh trong lần đầu khởi chạy khi dữ liệu hình ảnh chưa được sử dụng trước đó. Đây là trường hợp giúp quan sát hoạt động của ứng dụng khi phải thực hiện quá trình tải và hiển thị hình ảnh từ nguồn mạng.

Ở cả hai cấu hình, ứng dụng có thể tải và hiển thị danh sách sản phẩm đúng chức năng. Trong quá trình hình ảnh chưa tải hoàn tất, placeholder được hiển thị để duy trì bố cục giao diện và cung cấp phản hồi trực quan cho người dùng.

Kết quả từ Android Studio Profiler cho thấy mức sử dụng bộ nhớ có sự thay đổi trong quá trình hình ảnh được tải và giải mã. Tuy nhiên, giá trị tại một thời điểm cụ thể chỉ phản ánh trạng thái của ứng dụng tại thời điểm ghi nhận, do đó không được xem là giá trị trung bình cho toàn bộ quá trình thực thi.



Hình 9. Kết quả Profiler của TC01 – First Load trên Baseline



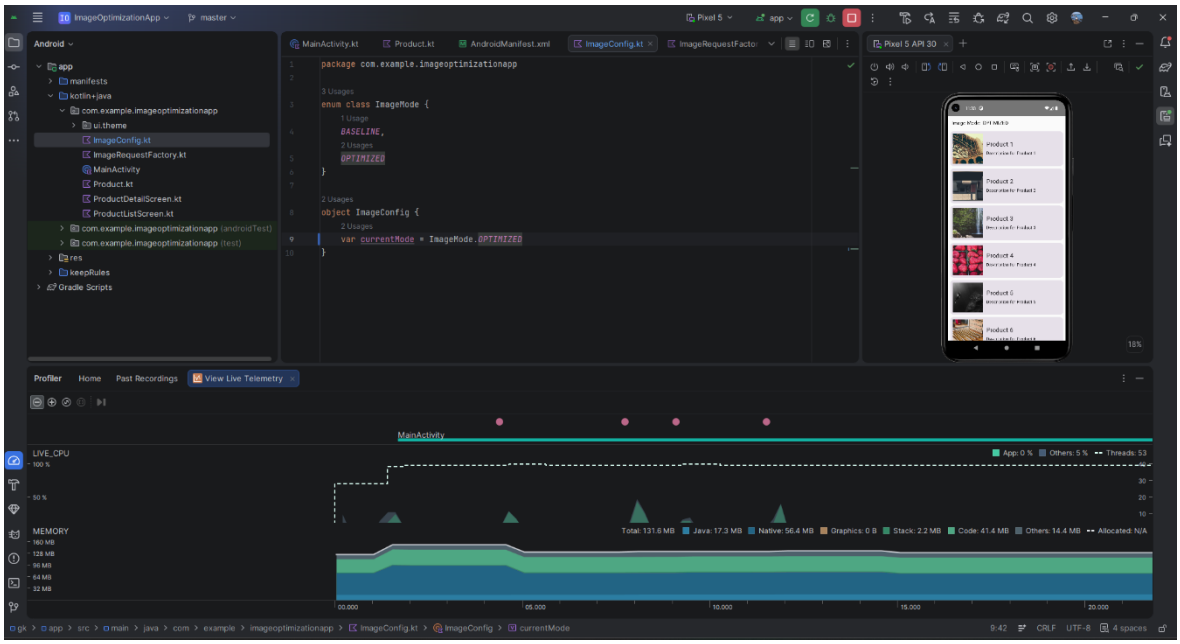
Hình 10. Kết quả Profiler của TC01 – First Load trên Optimized

5.1.2 TC02 – Cache Hit

TC02 được thực hiện nhằm đánh giá khả năng tái sử dụng hình ảnh sau khi dữ liệu đã được tải trước đó. Người dùng truy cập lại các hình ảnh đã hiển thị và quan sát thời gian phản hồi cũng như trạng thái của ứng dụng.

Ở cấu hình Optimized, Memory Cache và Disk Cache policy được khai báo tường minh trong ImageRequest. Vì vậy, những hình ảnh đã có trong bộ nhớ đệm có thể được tái sử dụng thay vì phải thực hiện lại toàn bộ quá trình tải từ nguồn mạng. Việc này có thể giúp hạn chế quá trình tải lại dữ liệu từ nguồn mạng và giảm sự phụ thuộc vào kết nối mạng trong các lần truy cập lại.

Kết quả kiểm thử cho thấy cơ chế cache đặc biệt có ý nghĩa trong trường hợp người dùng quay lại danh sách hoặc truy cập lại những hình ảnh đã được tải trước đó.



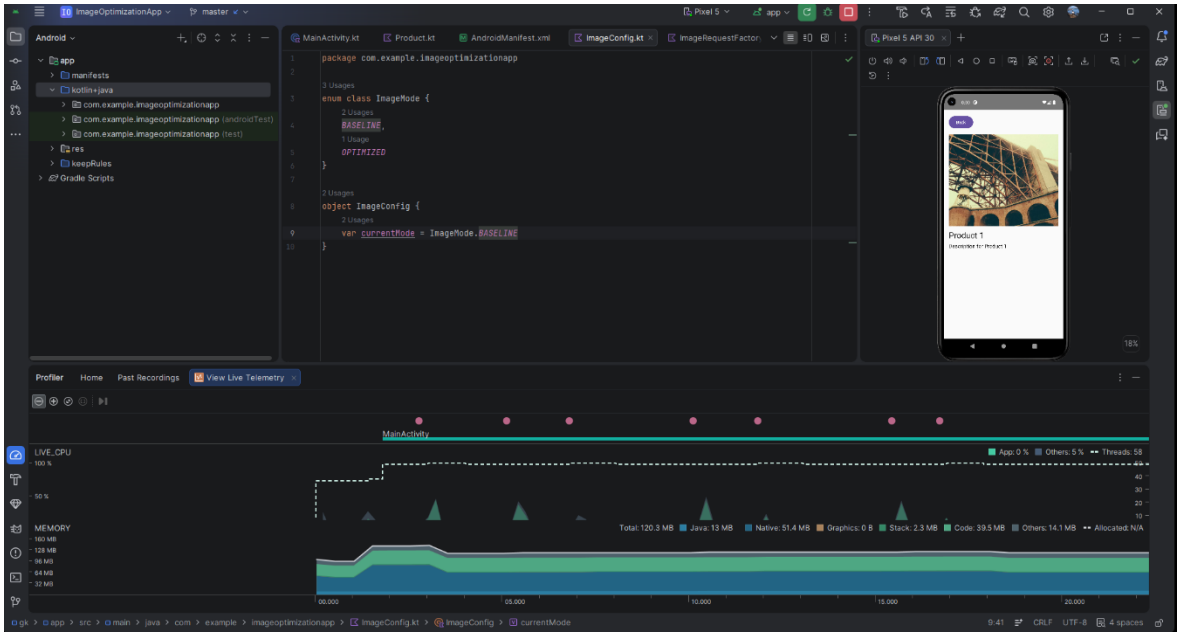
Hình 11. Kết quả kiểm thử TC02 – Cache Hit

5.1.3 TC03 và TC04 – Fast Scroll và High Resolution

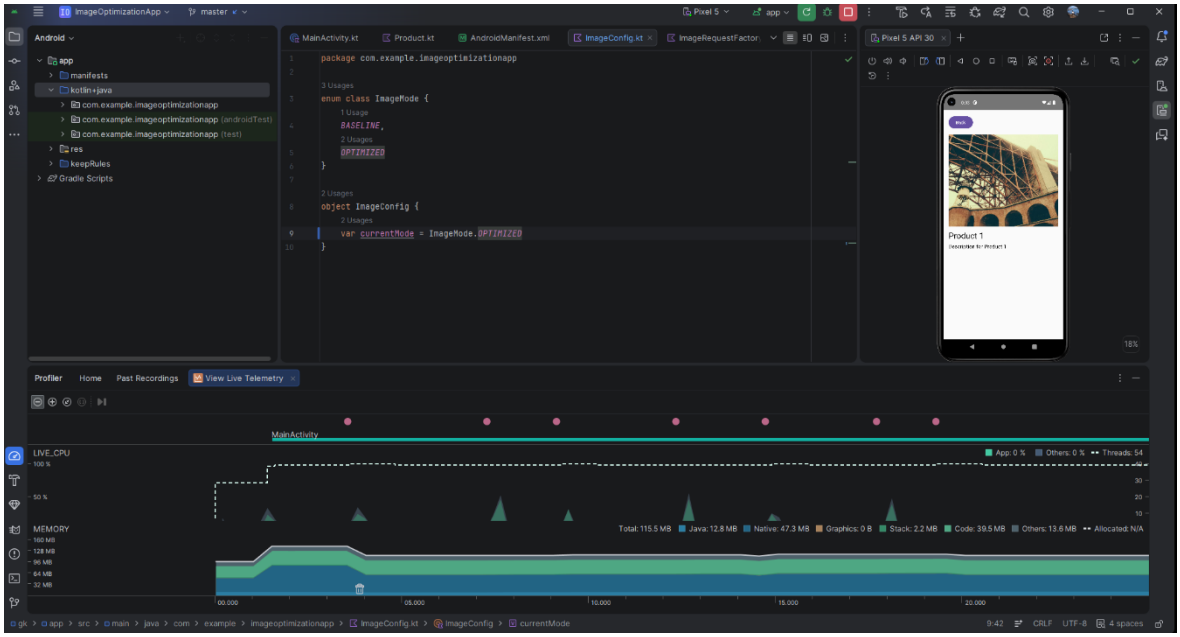
TC03 kiểm tra khả năng phản hồi của giao diện khi người dùng cuộn nhanh qua danh sách chứa nhiều hình ảnh. Việc sử dụng LazyColumn giúp ứng dụng chỉ tạo và hiển thị các thành phần cần thiết trong vùng giao diện hiện tại, thay vì hiển thị đồng thời toàn bộ danh sách.

Trong quá trình kiểm thử, cả Baseline và Optimized đều duy trì được chức năng cuộn danh sách. Các giá trị bộ nhớ ghi nhận bằng Profiler có thể dao động tùy theo thời điểm đo, trạng thái cache và các hình ảnh đang được giải mã. Vì vậy, một ảnh chụp Profiler riêng lẻ không đủ để kết luận cấu hình nào luôn sử dụng ít bộ nhớ hơn.

TC04 tiếp tục đánh giá việc hiển thị hình ảnh ở màn hình chi tiết. Tại thời điểm ghi nhận, cấu hình Baseline sử dụng khoảng **120,3 MB**, trong khi cấu hình Optimized ghi nhận khoảng **115,5 MB**. Chênh lệch khoảng **4,8 MB** cho thấy cấu hình Optimized có mức sử dụng bộ nhớ thấp hơn tại thời điểm đo. Kết quả này cho thấy Optimized có mức sử dụng bộ nhớ thấp hơn tại thời điểm đo; tuy nhiên, mức chênh lệch có thể chịu ảnh hưởng bởi nhiều yếu tố trong quá trình thực thi.



Hình 12. Kết quả kiểm thử TC03 và TC04 – Fast Scroll và High Resolution



Hình 13. Kết quả kiểm thử TC03 và TC04 – Fast Scroll và High Resolution

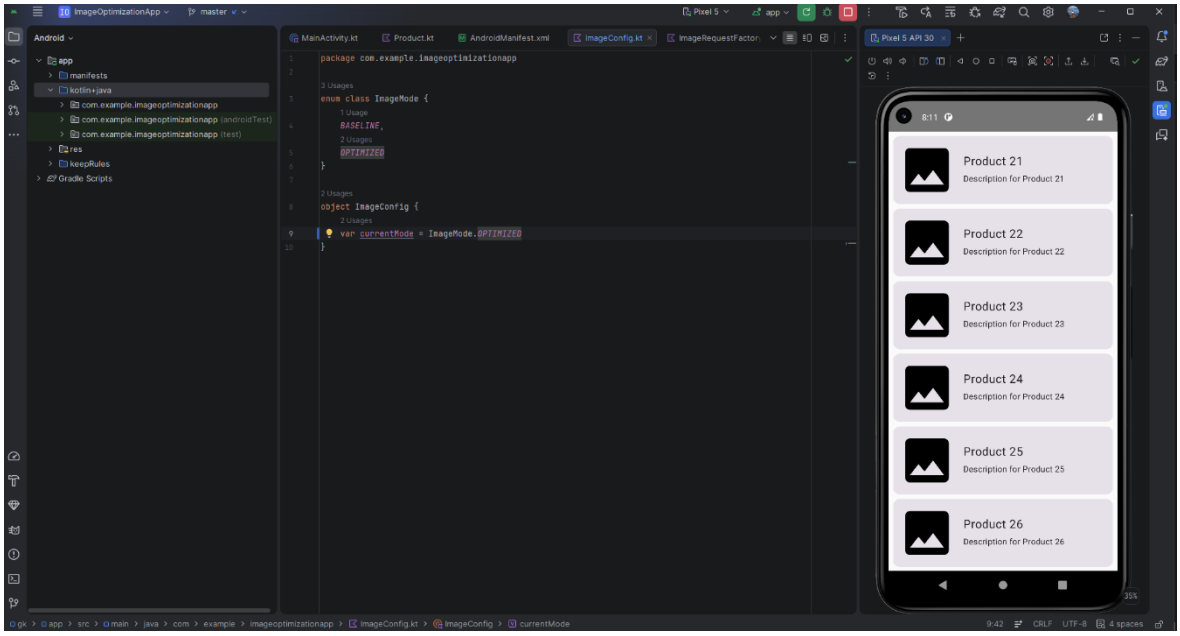
5.1.4 TC05 và TC06 – Offline và xử lý lỗi

TC05 được sử dụng để đánh giá hoạt động của ứng dụng khi không có kết nối mạng. Kết quả cho thấy các hình ảnh đã được tải và lưu vào cache trước đó có khả năng tiếp tục được sử dụng trong điều kiện offline. Ngược lại, những hình ảnh chưa được tải và chưa tồn tại trong cache không thể được lấy từ nguồn mạng khi thiết bị mất kết nối.

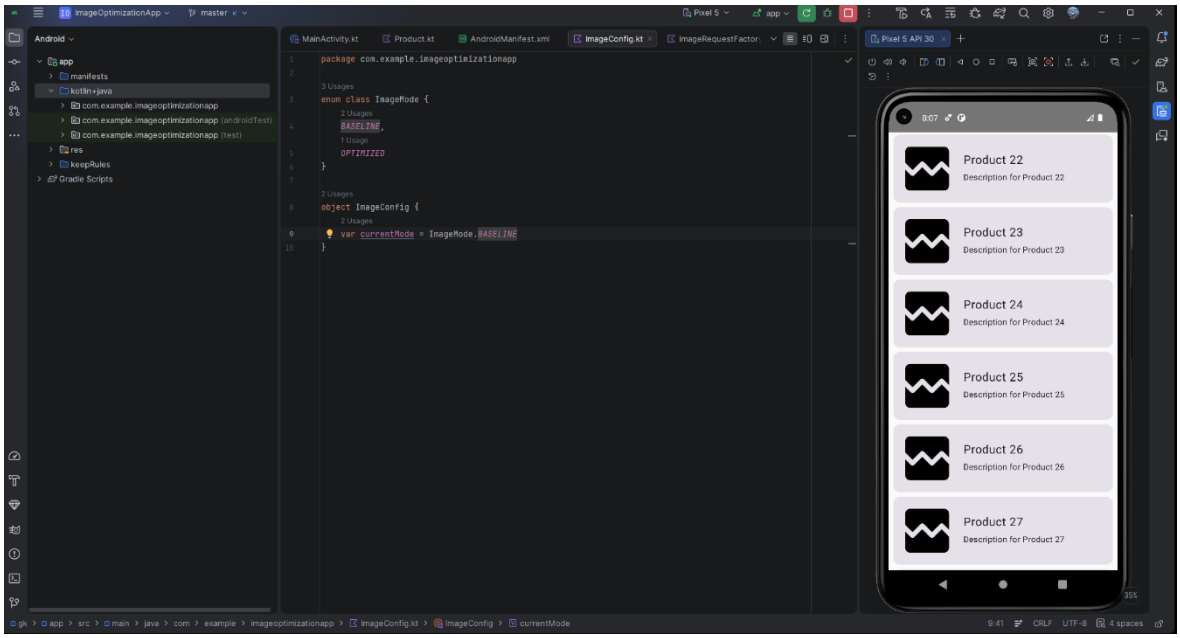
Kết quả này cho thấy cơ chế cache hỗ trợ khả năng tái sử dụng dữ liệu đã tải, nhưng không đồng nghĩa với việc ứng dụng có thể hiển thị mọi hình ảnh khi offline.

TC06 đánh giá khả năng xử lý URL hình ảnh không hợp lệ hoặc nguồn ảnh phản hồi chậm. Khi quá trình tải thất bại, ứng dụng sử dụng error placeholder thay vì để vùng hình ảnh trống hoặc làm gián đoạn giao diện. Khi nguồn ảnh phản hồi chậm, loading placeholder được hiển thị trong thời gian chờ và được thay thế bởi hình ảnh sau khi quá trình tải hoàn tất.

Như vậy, việc bổ sung loading và error placeholder không trực tiếp làm giảm mức sử dụng bộ nhớ nhưng giúp quá trình tải ảnh có trạng thái rõ ràng hơn và cải thiện trải nghiệm sử dụng.



Hình 14. Kết quả kiểm thử TC05 và TC06 – Offline và Error/Slow Network



Hình 15. Kết quả kiểm thử TC05 và TC06 – Offline và Error/Slow Network

5.1.5 TC07 – Dataset Scaling

TC07 đánh giá hoạt động của ứng dụng khi quy mô tập dữ liệu được thay đổi với các mức 100, 500 và 1000 sản phẩm. Mục tiêu của trường hợp kiểm thử này là

quan sát khả năng duy trì hoạt động của ứng dụng và sự thay đổi về mức sử dụng bộ nhớ khi số lượng phần tử trong danh sách tăng lên.

Kết quả ghi nhận từ Android Studio Profiler cho thấy với tập dữ liệu 100 sản phẩm, cấu hình Baseline sử dụng khoảng **130,4 MB**, trong khi Optimized sử dụng khoảng **117,4 MB**, thấp hơn **13,0 MB**, tương đương khoảng **10,0%** tại thời điểm đo.

Với 500 sản phẩm, Baseline ghi nhận khoảng **137,7 MB** và Optimized khoảng **136,4 MB**, mức chênh lệch chỉ **1,3 MB**. Khi tăng lên 1000 sản phẩm, Baseline ghi nhận khoảng **136,2 MB**, trong khi Optimized khoảng **128,7 MB**, thấp hơn khoảng **7,5 MB**, tương đương khoảng **5,5%**.

Quy mô dữ liệu	Baseline	Optimized	Chênh lệch	Mức giảm
100 sản phẩm	130,4 MB	117,4 MB	13,0 MB	~10,0%
500 sản phẩm	137,7 MB	136,4 MB	1,3 MB	~0,9%
1000 sản phẩm	136,2 MB	128,7 MB	7,5 MB	~5,5%

Kết quả cho thấy tại cả ba thời điểm ghi nhận, cấu hình Optimized đều có tổng mức sử dụng bộ nhớ thấp hơn Baseline. Tuy nhiên, mức chênh lệch không đồng đều giữa các quy mô dữ liệu. Mức giảm lớn nhất được ghi nhận ở tập 100 sản phẩm, khoảng 10,0%, trong khi ở 500 sản phẩm mức chênh lệch tương đối nhỏ.

Đồng thời, tổng bộ nhớ không tăng tuyến tính khi số lượng sản phẩm tăng từ 100 lên 500 và 1000. Điều này phù hợp với cách danh sách được triển khai bằng LazyColumn, trong đó các thành phần cần thiết cho vùng hiển thị được ưu tiên xử lý thay vì toàn bộ danh sách được hiển thị đồng thời.

Ngoài ra, số liệu Profiler còn chịu ảnh hưởng bởi trạng thái cache, quá trình garbage collection, các hình ảnh đang được giữ trong bộ nhớ và thời điểm thực hiện phép đo. Vì vậy, các giá trị trong bảng trên được xem là các **ảnh chụp trạng thái tại thời điểm kiểm thử**, không phải giá trị trung bình hay tỷ lệ cải thiện cố định của ứng dụng.

5.2 So sánh Baseline và Optimized

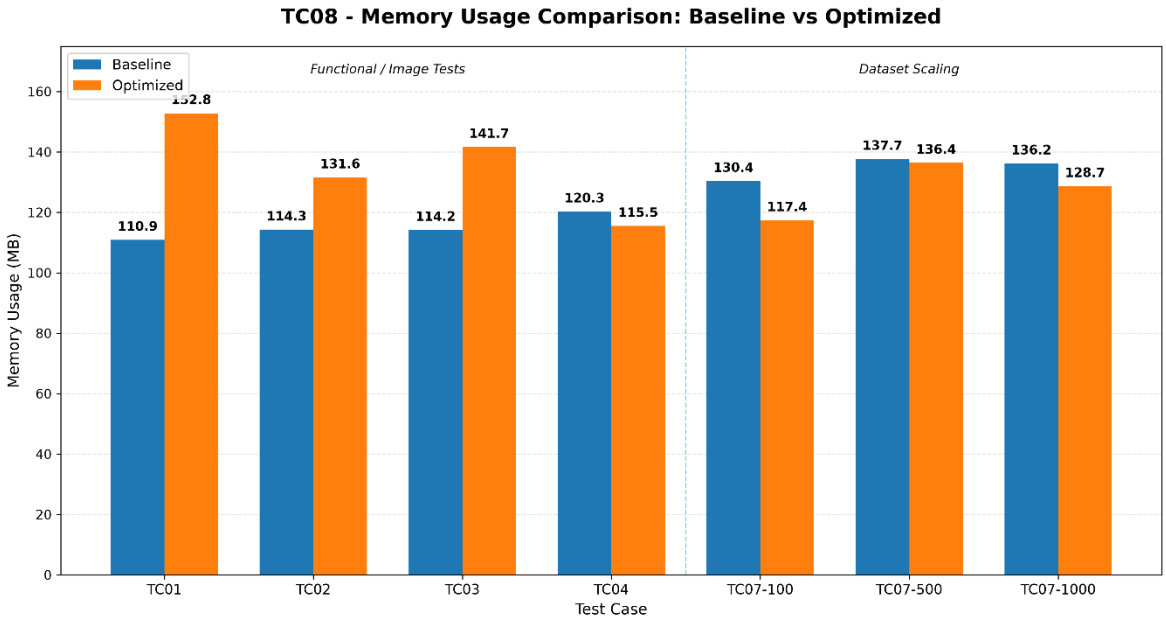
TC08 – Before/After Comparison được sử dụng để tổng hợp và đối chiếu kết quả giữa hai cấu hình Baseline và Optimized dựa trên các trường hợp kiểm thử đã thực hiện trước đó. TC08 không phải là một phép thử độc lập mà là bước tổng hợp các kết quả thu được nhằm đánh giá tác động tổng thể của cấu hình tối ưu hình ảnh.

Cấu hình Baseline chủ yếu sử dụng các thiết lập mặc định của Coil, trong khi Optimized sử dụng cấu hình ImageRequest tường minh hơn, bao gồm xác định kích thước thông qua size(width, height) và khai báo Memory Cache, Disk Cache policy. Hai cấu hình vẫn sử dụng cùng tập dữ liệu, giao diện và chức năng nhằm duy trì điều kiện so sánh tương đương.

Test Case	Mục tiêu	Điều kiện thực hiện	Baseline	Optimized	Chênh lệch	Đánh giá kết quả	Kết luận
TC01 Lần tải đầu tiên	Đánh giá mức sử dụng bộ nhớ khi mở ứng dụng và tải danh sách sản phẩm lần đầu.	Mở app, tải danh sách 100 sản phẩm, chụp Profiler khi danh sách hiển thị ổn định.	**110.9 MB**	**152.8 MB**	+41.9 MB (+37.8%)	Optimized có mức bộ nhớ cao hơn tại thời điểm chụp.	Cả hai đều hoạt động bình thường, không crash. Snapshot bộ nhớ của Optimized cao hơn do ảnh và cache được giữ trong bộ nhớ.
TC02 Cache Hit / Tải lại	Đánh giá khả năng sử dụng cache khi mở lại ứng dụng.	Đã mở app 1 lần trước, mở lại app, chụp Profiler khi danh sách hiển thị.	**114.3 MB**	**131.6 MB**	+17.3 MB (+15.1%)	Optimized có mức bộ nhớ cao hơn tại thời điểm chụp, nhưng ảnh được hiển thị gần như ngay lập tức.	Cache hoạt động đúng (ảnh hiển thị nhanh). Snapshot bộ nhớ cao hơn do dữ liệu ảnh được lưu trong memory cache.
TC03 Cuộn nhanh	Đánh giá mức sử dụng bộ nhớ khi cuộn nhanh danh sách sản phẩm.	Cuộn nhanh lên xuống danh sách nhiều lần, chụp Profiler tại thời điểm ổn định.	**114.2 MB**	**141.7 MB**	+27.5 MB (+24.1%)	Optimized có mức bộ nhớ cao hơn tại thời điểm chụp.	Cả hai đều xử lý tốt, không crash, không xảy ra OutOfMemoryError. Optimized vẫn duy trì trải nghiệm cuộn ổn định.
TC04 Ảnh độ phân giải cao	Đánh giá khả năng xử lý ảnh có độ phân giải cao.	Mở chi tiết sản phẩm có ảnh lớn (high-resolution), chụp Profiler.	**120.3 MB**	**115.5 MB**	−4.8 MB (−4.0%)	Optimized sử dụng ít bộ nhớ hơn.	Optimized xử lý ảnh lớn tốt hơn nhờ resize/downsampling, giảm mức sử dụng bộ nhớ.
TC05 Offline	Đánh giá khả năng hiển thị ảnh khi không có kết nối mạng.	Tắt mạng, mở danh sách và chi tiết sản phẩm (ảnh đã cache và chưa cache).	-	-	-	Ảnh đã cache hiển thị được. Ảnh chưa cache hiển thị placeholder lỗi.	Cache hoạt động đúng: chỉ các ảnh đã được tải trước đó mới hiển thị khi offline. Không load được ảnh chưa cache.
TC06 Lỗi / Mạng chậm	Đánh giá khả năng xử lý khi URL ảnh không hợp lệ hoặc mạng chậm.	Thử URL ảnh lỗi (bad URL), mô phỏng mạng chậm (throttle) và HTTP 503.	-	-	-	Hiển thị đúng placeholder loading và error.	Cả hai phiên bản đều xử lý lỗi và mạng chậm đúng, ứng dụng không bị crash, UI vẫn phản hồi bình thường.
TC07 – 100 sản phẩm	Đánh giá khả năng mở rộng khi tăng số lượng sản phẩm.	Tải danh sách 100 sản phẩm, chụp Profiler.	**130.4 MB**	**117.4 MB**	−13.0 MB (−10.0%)	Optimized sử dụng ít bộ nhớ hơn.	Optimized giảm mức sử dụng bộ nhớ khi danh sách có 100 sản phẩm.
TC07 – 500 sản phẩm	Đánh giá khả năng mở rộng khi tăng số lượng sản phẩm.	Tải danh sách 500 sản phẩm, chụp Profiler.	**137.7 MB**	**136.4 MB**	−1.3 MB (−0.9%)	Optimized sử dụng ít bộ nhớ hơn.	Optimized giảm nhẹ mức sử dụng bộ nhớ khi danh sách có 500 sản phẩm.
TC07 – 1000 sản phẩm	Đánh giá khả năng mở rộng khi tăng số lượng sản phẩm.	Tải danh sách 1000 sản phẩm, chụp Profiler.	**136.2 MB**	**128.7 MB**	−7.5 MB (−5.5%)	Optimized sử dụng ít bộ nhớ hơn.	Optimized giảm mức sử dụng bộ nhớ khi danh sách có 1000 sản phẩm.

Hình 16. Tổng hợp kết quả kiểm thử giữa Baseline và Optimized

Kết quả tổng hợp cho thấy sự khác biệt giữa hai cấu hình thay đổi tùy theo từng trường hợp kiểm thử. Ở một số tình huống, mức sử dụng bộ nhớ giữa Baseline và Optimized không có sự chênh lệch lớn. Tuy nhiên, khi hiển thị hình ảnh ở màn hình chi tiết hoặc trong một số phép thử với tập dữ liệu lớn, Optimized ghi nhận mức sử dụng bộ nhớ thấp hơn Baseline tại thời điểm đo.



Hình 17. So sánh mức sử dụng bộ nhớ giữa Baseline và Optimized

Đối với cache, cả hai cấu hình đều sử dụng Coil nên Baseline vẫn có thể thừa hưởng cơ chế cache mặc định của thư viện. Điểm khác biệt của Optimized là các Memory Cache và Disk Cache policy được khai báo tường minh trong ImageRequest. Kết quả TC02 và TC05 cho thấy những hình ảnh đã được tải trước đó có khả năng được tái sử dụng trong các điều kiện cache phù hợp, trong khi những hình ảnh chưa có dữ liệu cache vẫn cần kết nối mạng để tải.

Kết quả TC07 cũng cho thấy Optimized có mức sử dụng bộ nhớ thấp hơn Baseline tại cả ba thời điểm ghi nhận với tập dữ liệu 100, 500 và 1000 sản phẩm. Tuy nhiên, mức chênh lệch không đồng đều giữa các quy mô dữ liệu. Do đó, các số liệu này được xem là kết quả tại thời điểm thực nghiệm thay vì một tỷ lệ cải thiện cố định của cấu hình Optimized.

Nhìn chung, kết quả TC08 cho thấy sự khác biệt giữa Baseline và Optimized không nên được đánh giá chỉ dựa trên một giá trị bộ nhớ riêng lẻ. Việc đánh giá cần kết hợp khả năng kiểm soát kích thước ImageRequest, hành vi cache, mức sử dụng bộ nhớ và khả năng duy trì hoạt động của ứng dụng trong các kịch bản kiểm thử khác nhau. Các tác động này được phân tích cụ thể hơn trong phần tiếp theo.

5.3 Phân tích tác động của các kỹ thuật tối ưu

Trong cấu hình Optimized, kích thước ImageRequest được xác định thông qua size(width, height), giúp chủ động kiểm soát kích thước hình ảnh cần xử lý theo vùng hiển thị. Kết quả TC04 cho thấy Optimized có mức sử dụng bộ nhớ thấp hơn Baseline tại thời điểm đo khi hiển thị hình ảnh ở màn hình chi tiết. Tuy nhiên, kết quả này chỉ phản ánh trạng thái ứng dụng tại thời điểm thực nghiệm và không được xem là tác động riêng của kỹ thuật resize.

Đối với cache, cả hai cấu hình đều sử dụng Coil và có thể sử dụng cơ chế cache của thư viện. Điểm khác biệt là Optimized khai báo tường minh Memory Cache và Disk Cache policy trong ImageRequest. Kết quả TC02 và TC05 cho thấy dữ liệu hình ảnh đã được tải trước đó có khả năng được tái sử dụng trong điều kiện phù hợp, trong khi ảnh chưa có trong cache vẫn cần kết nối mạng để tải.

Bên cạnh đó, LazyColumn hỗ trợ xử lý danh sách lớn bằng cách tập trung vào các thành phần cần thiết cho vùng hiển thị. Nhìn chung, kết quả thực nghiệm phản ánh tác động tổng hợp của kích thước ImageRequest, cache, LazyColumn và trạng thái runtime thay vì một kỹ thuật riêng lẻ.

5.4 Đánh giá tổng hợp và thảo luận

Qua các trường hợp kiểm thử, phiên bản Optimized vẫn duy trì đầy đủ các chức năng đã có ở Baseline, đồng thời sử dụng các cấu hình tường minh hơn trong quá trình tải và xử lý hình ảnh. Ứng dụng vẫn đảm bảo Product List, Product Detail, Loading/Error placeholder và khả năng làm việc với các tập dữ liệu có quy mô khác nhau.

Kết quả Profiler cho thấy mức chênh lệch bộ nhớ không đồng đều trong mọi trường hợp. Ở một số thời điểm, hai cấu hình có mức sử dụng bộ nhớ gần nhau, trong khi ở các trường hợp khác, Optimized ghi nhận mức bộ nhớ thấp hơn. Ví dụ, tại TC07 với 1000 sản phẩm, mức chênh lệch ghi nhận khoảng 7,5 MB, tương đương khoảng 5,5%.

Điều này cho thấy không nên đánh giá hiệu quả tối ưu chỉ bằng một ảnh chụp Profiler duy nhất. Bộ nhớ của ứng dụng Android còn chịu ảnh hưởng bởi garbage collection, trạng thái cache, hình ảnh đang được giữ trong bộ nhớ và thời điểm thực hiện phép đo. Vì vậy, các kết quả trong báo cáo được xem là bằng chứng thực nghiệm trong môi trường kiểm thử đã xác định, không phải mức cải thiện cố định cho mọi thiết bị và mọi điều kiện.

Từ các kết quả thu được, phiên bản Optimized đáp ứng mục tiêu chính của đề tài: duy trì đầy đủ chức năng của ứng dụng, chủ động kiểm soát kích thước ImageRequest và khai báo tường minh các chính sách cache trong quá trình tải hình ảnh. Kết quả thực nghiệm cũng cho thấy ứng dụng duy trì hoạt động ổn định trong các kịch bản đã kiểm thử. Đây là cơ sở để lựa chọn Optimized làm cấu hình cuối cùng cho sản phẩm được trình bày trong Chương 6.

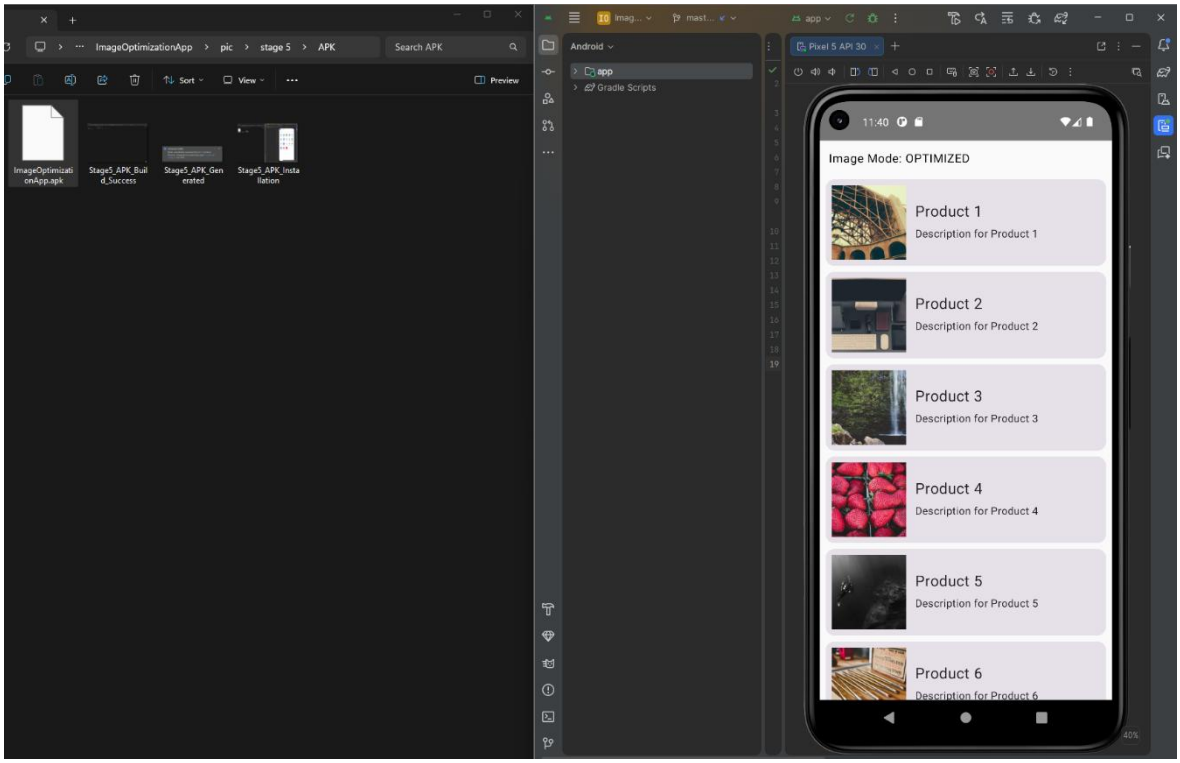
CHƯƠNG 6. SẢN PHẨM VÀ KẾT LUẬN

6.1 Sản phẩm hoàn thiện

Sau quá trình phân tích, xây dựng, tối ưu và thực nghiệm, đề tài đã hoàn thiện ứng dụng Android **ImageOptimizationApp** với mục tiêu nghiên cứu và đánh giá các phương pháp tối ưu quá trình tải, lưu trữ đệm và hiển thị hình ảnh.

Ứng dụng được phát triển bằng Kotlin và Jetpack Compose, sử dụng Coil để tải và hiển thị hình ảnh từ nguồn trực tuyến. Các chức năng chính bao gồm hiển thị danh sách sản phẩm, xem thông tin chi tiết, xử lý trạng thái Loading và Error, đồng thời hỗ trợ các cơ chế tối ưu như giới hạn kích thước hình ảnh, Memory Cache và Disk Cache.

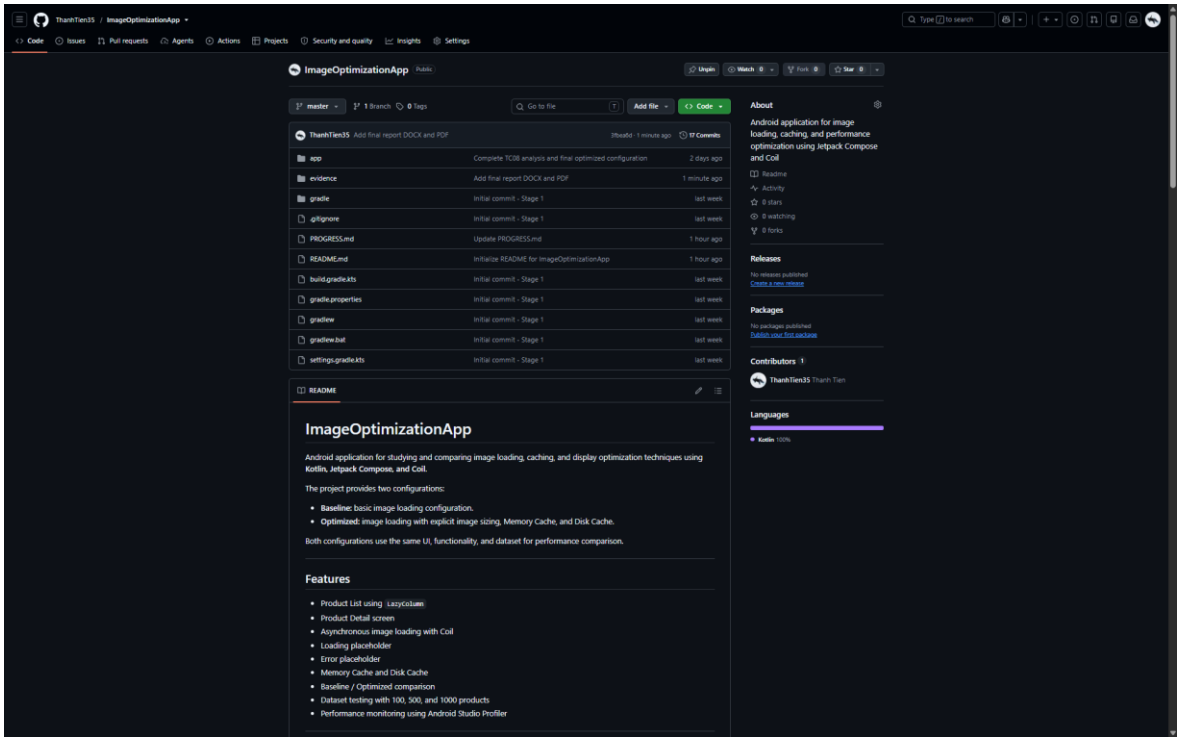
Phiên bản cuối cùng sử dụng cấu hình Optimized và đã được build thành tệp ImageOptimizationApp.apk. Tệp APK được kiểm tra thông qua quá trình cài đặt và chạy thử nhằm đảm bảo các chức năng chính của ứng dụng hoạt động bình thường.



Hình 18. Tệp APK và kết quả chạy thử phiên bản hoàn thiện của ứng dụng

Toàn bộ mã nguồn và quá trình phát triển của đề tài được quản lý trên GitHub. Repository không chỉ lưu trữ mã nguồn trong thư mục app/ mà còn bao gồm thư mục evidence/, tệp PROGRESS.md và README.md.

Thư mục evidence/ lưu trữ các minh chứng theo từng giai đoạn phát triển và kiểm thử. Stage 1 đến Stage 3 thể hiện quá trình xây dựng và tối ưu ứng dụng; Stage 4 lưu trữ minh chứng của TC01–TC07; Stage 5 chứa APK, cấu hình cuối cùng, tài liệu liên quan đến báo cáo và kết quả tổng hợp TC08. Tệp PROGRESS.md ghi nhận tiến độ và các công việc đã thực hiện theo từng giai đoạn của nhóm, trong khi README.md cung cấp thông tin tổng quan và hướng dẫn cần thiết để chạy project.



Hình 19. Repository của dự án ImageOptimizationApp trên GitHub

Repository: *https://github.com/ThanhTien35/ImageOptimizationApp*

Cách tổ chức này giúp người xem có thể kiểm tra mã nguồn, sản phẩm cuối cùng, các minh chứng thực nghiệm và quá trình thực hiện đề tài của nhóm một cách thuận tiện.

6.2 Hướng dẫn cài đặt và sử dụng

Ứng dụng có thể được chạy trực tiếp từ mã nguồn hoặc cài đặt thông qua tệp APK.

Đối với mã nguồn, người dùng tải hoặc clone repository về máy tính, mở project **ImageOptimizationApp** bằng Android Studio và chờ Gradle hoàn tất quá trình đồng bộ. Sau đó, lựa chọn thiết bị Android hoặc emulator phù hợp và sử dụng chức năng Run để build và khởi chạy ứng dụng.

Đối với APK, người dùng có thể cài đặt trực tiếp tệp ImageOptimizationApp.apk trên thiết bị Android tương thích. Sau khi cài đặt, ứng dụng có thể được khởi động mà không cần sử dụng Android Studio.

Khi ứng dụng hoạt động, màn hình chính hiển thị danh sách sản phẩm cùng hình ảnh tương ứng. Người dùng có thể cuộn danh sách và lựa chọn sản phẩm để xem màn hình chi tiết. Trong quá trình tải ảnh, Loading placeholder được hiển thị khi dữ liệu chưa sẵn sàng; nếu quá trình tải thất bại, Error placeholder được sử dụng để duy trì trạng thái giao diện.

6.3 Hướng dẫn tái hiện thực nghiệm

Project duy trì hai cấu hình **Baseline** và **Optimized** trong cùng mã nguồn nhằm phục vụ quá trình kiểm thử và so sánh. Cấu hình cần sử dụng được lựa chọn thông qua ImageConfig.currentMode.

Để tái hiện quá trình thực nghiệm, thực hiện theo các bước sau:

1. Mở project bằng Android Studio.
2. Lựa chọn ImageMode.BASELINE hoặc ImageMode.OPTIMIZED.
3. Thiết lập quy mô dữ liệu cần kiểm thử: 100, 500 hoặc 1000 sản phẩm.
4. Build và chạy ứng dụng trên cùng thiết bị hoặc emulator.
5. Sử dụng Android Studio Profiler để theo dõi CPU và Memory.
6. Thực hiện trường hợp kiểm thử tương ứng từ TC01 đến TC07.
7. Ghi nhận kết quả, sau đó chuyển sang cấu hình còn lại và thực hiện lại trong điều kiện tương đương.
8. Tổng hợp và đối chiếu kết quả giữa Baseline và Optimized thông qua TC08.

Các minh chứng tương ứng với từng trường hợp kiểm thử được lưu trong thư mục evidence/ của repository. Kết quả Profiler có thể dao động tùy thuộc vào trạng thái cache, garbage collection, kết nối mạng và thời điểm thực hiện phép đo. Vì vậy, việc tái hiện cần duy trì môi trường và điều kiện kiểm thử tương đương.

6.4 Kết quả đạt được và hạn chế

Đề tài đã hoàn thành ứng dụng Android với các chức năng tải và hiển thị hình ảnh, Product List, Product Detail, Loading/Error placeholder và hai cấu hình Baseline, Optimized phục vụ quá trình thực nghiệm.

Thông qua TC01–TC08, ứng dụng đã được kiểm thử trong các trường hợp tải ảnh lần đầu, cache hit, cuộn nhanh, hiển thị ảnh kích thước lớn, offline, URL lỗi và thay đổi quy mô dữ liệu. Kết quả cho thấy Optimized duy trì đầy đủ chức năng và ghi nhận mức sử dụng bộ nhớ thấp hơn Baseline tại một số thời điểm thực nghiệm.

Bên cạnh kết quả đạt được, đề tài vẫn còn một số hạn chế. Các số liệu Profiler chủ yếu là giá trị tại thời điểm ghi nhận trên một môi trường kiểm thử và chưa phải giá trị trung bình từ benchmark tự động. Các chỉ số như thời gian tải/hiển thị và số lượng jank frame chưa được đo định lượng. Ngoài ra, các trường hợp xóa hoàn toàn cache, khởi động lại ứng dụng và thay đổi nội dung hình ảnh trên máy chủ chưa được tách thành các phép thử độc lập.

Nhìn chung, đề tài đã hoàn thành mục tiêu xây dựng và đánh giá ứng dụng Android có khả năng tải, lưu trữ đệm và hiển thị hình ảnh. Cấu hình Optimized cho phép chủ động kiểm soát kích thước ImageRequest và khai báo tường minh các chính sách cache, đồng thời duy trì hoạt động ổn định trong các kịch bản đã kiểm thử. Mã nguồn, APK và các minh chứng thực nghiệm được lưu trữ trên GitHub để phục vụ việc kiểm tra và tái hiện kết quả.

TÀI LIỆU THAM KHẢO

Android Developers. (2026). *Jetpack Compose*. Google. https://developer.android.com/compose

Android Developers. (2026). *Lists and grids*. Google. https://developer.android.com/develop/ui/compose/lists

Android Developers. (2026). *App performance guide*. Google. https://developer.android.com/topic/performance

Android Developers. (2026). *Profile your app performance*. Google. https://developer.android.com/studio/profile

Coil Contributors. (2026). *Coil – Image loading for Android and Compose Multiplatform*. https://coil-kt.github.io/coil/

Coil Contributors. (2026). *Compose – Coil*. https://coil-kt.github.io/coil/compose/

Coil Contributors. (2026). *ImageRequest – Coil API Documentation*. https://coil-kt.github.io/coil/api/coil-core/coil3.request/-image-request/

Kotlin Foundation. (2026). *Kotlin Documentation*. https://kotlinlang.org/docs/home.html